

	DEPARTMENT OF COMPUTER ENGINEERING (DCE) SOC DESIGN MMH: SOCD334264
Group: Anh em vi mạch K24 Name: 1. Lê Quốc Khánh - 24161276 2. Đặng Quang Được - 24161217 3. Lê Minh Hải - 24161225 4. Nguyễn Mai Trung Dũng - 24161206 5. Nguyễn Hữu Đức - 24161221 6. Lã Phan Ân Kỳ - 24161294	

BÁO CÁO LAB 1 THIẾT KẾ HỆ THỐNG ZYNQ SOC

Giới thiệu bài Lab

Bài Lab hướng dẫn xây dựng một hệ thống Zynq SoC cơ bản theo quy trình Hardware/Software Codesign. Phần cứng được thiết kế trong Vivado IP Integrator, trong đó Processing System (PS) của Zynq giao tiếp với AXI GPIO đặt trong Programmable Logic (PL) thông qua giao tiếp AXI. Sau khi hoàn thành thiết kế phần cứng, hệ thống được tổng hợp, triển khai, tạo bitstream và xuất hardware platform để phát triển ứng dụng C trên ARM Cortex-A9 trong Vitis.

Tutorial 1 gốc sử dụng Vivado 2014.1, Xilinx SDK và ZedBoard. Trong bài thực hành này, quy trình được chuyển sang Vivado/Vitis 2026.1 và board preset Zynq-7000 ZC706 (xc7z045ffg900-2). Target language được chọn là Verilog. Do yêu cầu thực hiện theo hướng simulation-only và không có board ZC706 vật lý kết nối qua JTAG, phần cuối được kiểm chứng bằng kết quả build ứng dụng bare-metal và mô phỏng hành vi LED 4-bit; báo cáo không khẳng định đã nạp chương trình xuống board thật.

Mục tiêu bài Lab

- Hiểu kiến trúc Zynq SoC gồm Processing System (PS) và Programmable Logic (PL), cũng như cách hai phần giao tiếp thông qua bus AXI.

- Làm quen với Vivado IP Integrator, Block Design, Address Editor, HDL Wrapper, RTL Analysis, Synthesis, Implementation và Generate Bitstream.
- Kết nối AXI GPIO với cổng M_AXI_GP0 của ARM Cortex-A9 thông qua AXI SmartConnect và ánh xạ ngoại vi vào không gian địa chỉ của PS.
- Xuất hardware platform XSA và xây dựng platform/application bare-metal trong Vitis Embedded Development.
- Viết và build chương trình C sử dụng driver XGpio để điều khiển LED, đồng thời kiểm chứng chuỗi trạng thái LED bằng Behavioral Simulation.

Công cụ cần thiết

Phần cứng / board preset

- Zynq-7000 ZC706 Evaluation Board được chọn làm board preset trong Vivado; bài thực hành hiện tại không sử dụng board vật lý ở bước Run.

Phần mềm

- AMD Vivado 2026.1.
- AMD Vitis Unified IDE 2026.1 và Vitis Embedded Development 2026.1.
- Target language: Verilog; application bare-metal chạy trên processor ps7_cortexa9_0 với operating system standalone.

Kết quả đạt được

- Xây dựng thành công Block Design Zynq PS - AXI SmartConnect - AXI GPIO - LED 4-bit.
- AXI GPIO được ánh xạ tại địa chỉ 0x41200000 với vùng địa chỉ 64 KB.
- Validate Design, RTL Analysis, Synthesis, Implementation và Generate Bitstream hoàn thành; timing implementation không có failing endpoint.
- Xuất thành công hardware platform first_zynq_system_wrapper.xsa có kèm bitstream.
- Tạo platform standalone cho ps7_cortexa9_0, tạo ứng dụng led_control_app và build thành công file led_control_app.elf.
- Mô phỏng Behavioral Simulation cho chuỗi LED 4-bit 0001 → 0010 → 0100 → 1000 và lặp lại.

Phần công việc thực hiện

A. Câu hỏi

1. Giải thích sự khác nhau giữa Processing System (PS) và Programmable Logic (PL).

Processing System (PS) là phần xử lý cứng tích hợp sẵn trong Zynq, chứa ARM Cortex-A9, các bộ điều khiển bộ nhớ, clock/reset và nhiều ngoại vi chuẩn. PS thực thi chương trình C và đóng vai trò AXI master khi truy cập các ngoại vi được ánh xạ vào không gian địa chỉ. Programmable Logic (PL) là vùng FPGA có thể cấu hình để xây dựng các IP phần cứng tùy biến. Trong bài Lab, AXI GPIO nằm trong PL, còn chương trình điều khiển chạy trên ARM Cortex-A9 thuộc PS.

2. Vai trò của AXI Interconnect trong hệ thống Zynq là gì?

AXI Interconnect có nhiệm vụ nối AXI master và AXI slave, định tuyến địa chỉ và dữ liệu, đồng thời hỗ trợ các chức năng kết nối cần thiết giữa nhiều thiết bị trên bus. Tutorial gốc sử dụng AXI Interconnect. Trong Vivado 2026.1, Connection Automation của bài thực hành tạo AXI SmartConnect để thực hiện vai trò tương đương giữa `processing_system7_0/M_AXI_GP0` và `axi_gpio_0/S_AXI`. Mã nguồn AXI4 Interconnect do giảng viên cung cấp được xem như tài liệu tham khảo ở mức RTL; bài Lab không thay AXI SmartConnect bằng repository này.

3. Tại sao cần Processor System Reset IP?

Processor System Reset IP tạo các tín hiệu reset đã được đồng bộ theo clock hệ thống và phân phối reset phù hợp cho interconnect, peripheral và logic trong PL. Khi hệ thống khởi động hoặc reset, các khối AXI phải được đưa về trạng thái xác định trước khi trao đổi giao dịch. Trong thiết kế này, Connection Automation tự động tạo khối `rst_ps7_0_50M` và kết nối reset cho AXI SmartConnect và AXI GPIO.

4. AXI GPIO hoạt động như thế nào trong bài Lab này?

AXI GPIO là một AXI slave memory-mapped đặt trong PL. ARM Cortex-A9 truy cập thanh ghi của AXI GPIO thông qua `M_AXI_GP0` và AXI SmartConnect. Trong Address Editor, ngoại vi được gán base address `0x41200000`. Chương trình C khởi tạo driver `XGpio`, đặt channel 1 ở chế độ output bằng `XGpio_SetDataDirection(..., 0x00)` và dùng `XGpio_DiscreteWrite` để ghi mẫu bit ra LED 4-bit.

5. Trình bày quy trình từ Block Design đến Generate Bitstream.

Quy trình gồm: tạo Block Design; thêm ZYNQ7 Processing System; Run Block Automation để áp dụng board preset; thêm AXI GPIO; Run Connection Automation để nối AXI và tạo SmartConnect/reset/clock; nối GPIO ra LED; kiểm tra Address Editor; Validate Design; Create HDL Wrapper; kiểm tra RTL; Run Synthesis; Run Implementation; kiểm tra timing và utilization; cuối cùng Generate Bitstream. Sau đó hardware platform được export thành XSA để tiếp tục phần mềm trong Vitis.

B. Thực hành

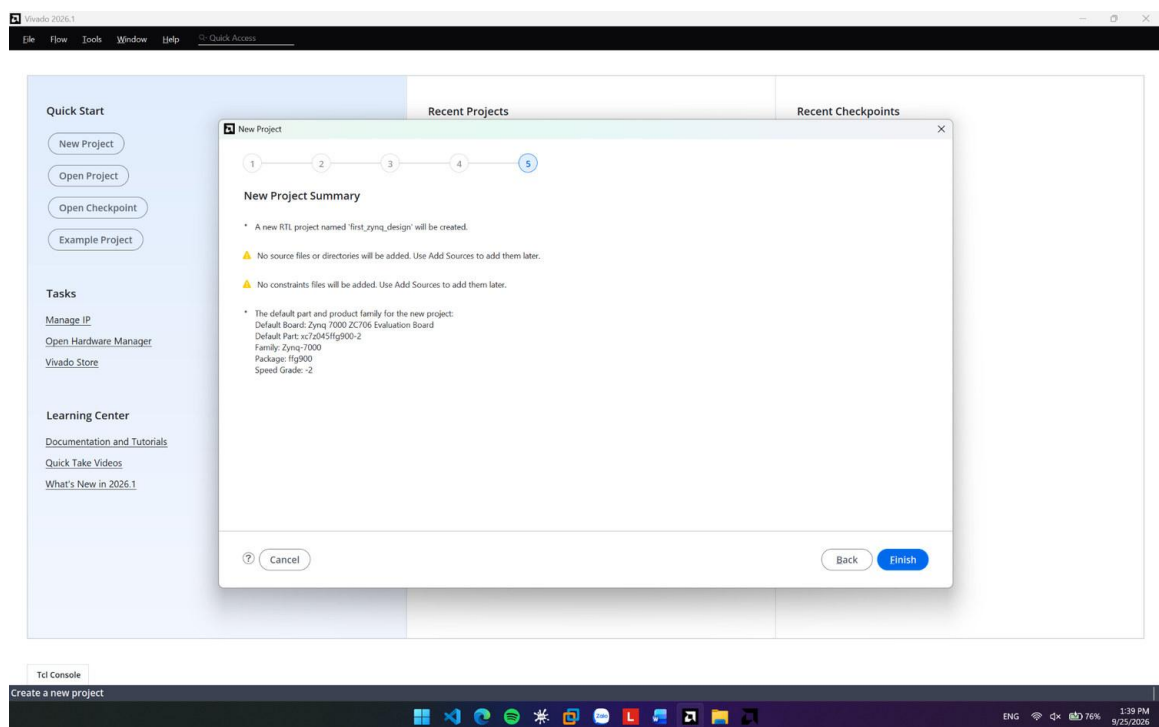
Exercise 1A - Tạo project Vivado

Bước 1 - Tạo project first_zynq_design

Cơ sở lý thuyết: Trong Vivado, project là đơn vị quản lý toàn bộ nguồn thiết kế, IP, ràng buộc chân, cấu hình thiết bị và các kết quả synthesis/implementation. Việc chọn đúng target device/board ngay từ đầu rất quan trọng vì thông tin này quyết định kiến trúc FPGA, tài nguyên logic, preset phần cứng và các interface mà Vivado có thể tự động cấu hình. Với Zynq-7000, phần PS và PL nằm trên cùng một chip nhưng vẫn được công cụ quản lý như hai miền chức năng khác nhau.

Mục đích của bước: Tạo một môi trường thiết kế thống nhất cho toàn bộ Lab và khai báo đúng ZC706 làm nền phần cứng để các bước IP Integrator, synthesis và implementation về sau sử dụng đúng device xc7z045ffg900-2.

Thao tác thực hiện: Khởi động Vivado 2026.1, chọn Create Project và đặt tên project là first_zynq_design. Chọn RTL Project, chưa cần thêm source RTL ở bước đầu. Ở phần Project Settings, chọn Target language là Verilog và Simulator language là Mixed. Tại trang chọn board, chọn Zynq 7000 ZC706 Evaluation Board, tương ứng device xc7z045ffg900-2. Sau đó kiểm tra trang Project Summary và nhấn Finish.



Hình 1. Project Summary khi tạo project first_zynq_design.

Kết quả và ý nghĩa: Xác nhận các tham số nền của project trước khi đi vào IP Integrator. Việc chọn đúng board/device rất quan trọng vì board preset quyết định cấu hình chân, DDR, FIXED_IO và các interface board có thể sử dụng ở những bước sau. Khác tutorial gốc dùng ZedBoard, bài này dùng ZC706 nhưng vẫn thuộc họ Zynq-7000 nên quy trình PS-PL không thay đổi về bản chất.

Kiến thức rút ra: Qua bước này có thể hiểu rằng một dự án SoC không bắt đầu từ code C hay Verilog riêng lẻ mà bắt đầu từ việc xác định đúng nền tảng phần cứng. Chọn sai board/device có thể làm preset PS, chân I/O hoặc kết quả implementation không còn phù hợp.

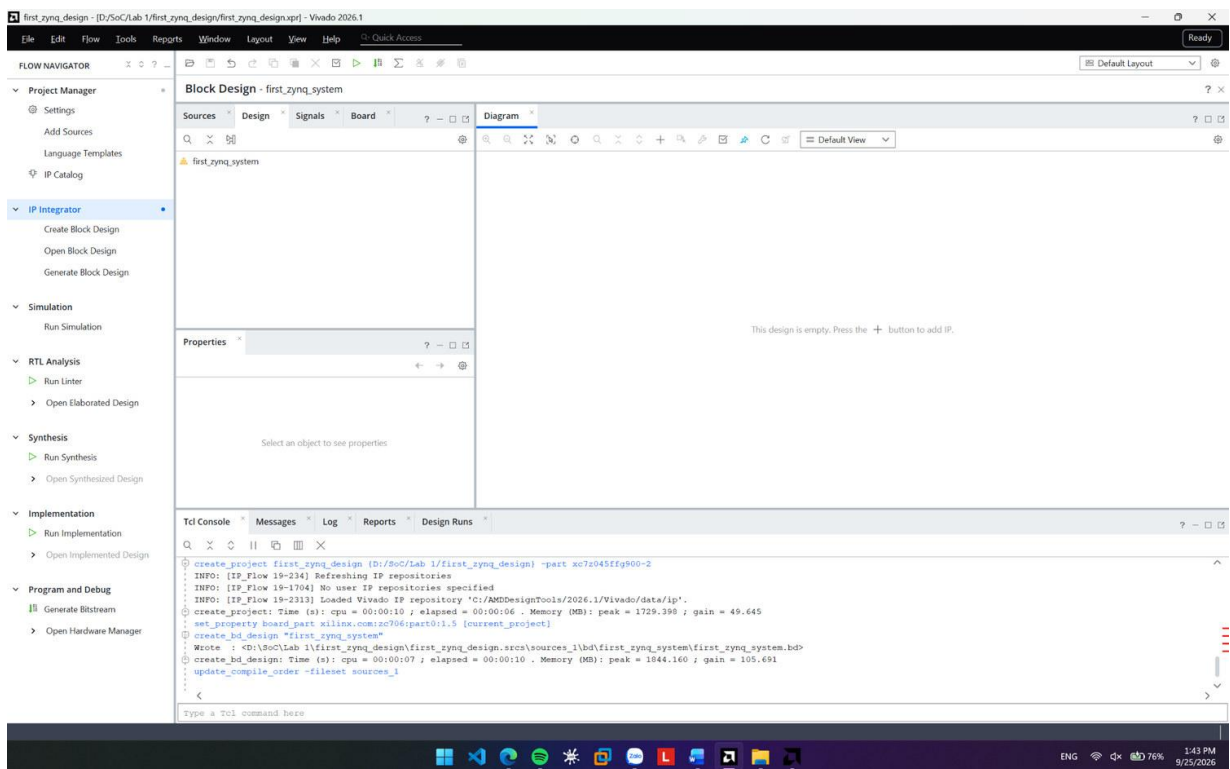
Exercise 1B - Xây dựng hệ thống Zynq trong Vivado IP Integrator

Bước 1 - Tạo Block Design

Cơ sở lý thuyết: IP Integrator cho phép xây dựng hệ thống ở mức khối thay vì phải tự nối hàng trăm tín hiệu AXI, clock và reset bằng RTL. Một Block Design mô tả quan hệ giữa các IP core, interface bus và các cổng ngoài; từ đó Vivado có thể sinh HDL wrapper và các file cấu hình cần thiết. Đây là tư duy thiết kế SoC theo kiến trúc, trong đó mỗi IP là một khối chức năng độc lập có giao tiếp chuẩn.

Mục đích của bước: Tạo không gian kiến trúc để ghép Processing System, AXI interconnect, AXI GPIO, reset và LED thành một hệ thống Zynq hoàn chỉnh.

Thao tác thực hiện: Từ Flow Navigator, chọn IP Integrator → Create Block Design. Đặt tên Block Design là `first_zynq_system`. Khi tạo xong, Vivado mở canvas trống để thêm các IP và xây dựng kết nối hệ thống.



Hình 2. Block Design `first_zynq_system` ở trạng thái ban đầu.

Kết quả và ý nghĩa: Canvas trống là nơi mô tả kiến trúc phần cứng ở mức khối. Ở thời điểm này chưa có PS, AXI hay ngoại vi. Việc tạo Block Design riêng giúp Vivado quản lý cấu hình IP, clock/reset và các connection automation một cách trực quan.

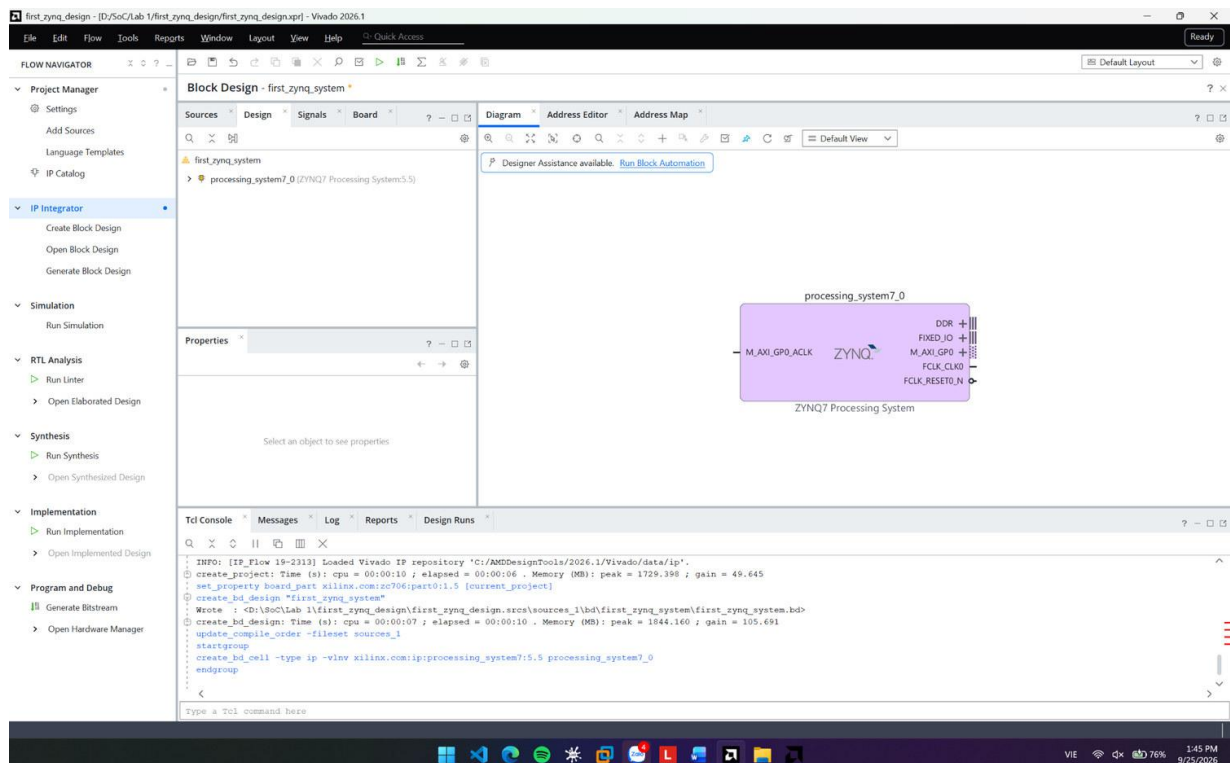
Kiến thức rút ra: Bước này giúp phân biệt thiết kế RTL thuần với thiết kế SoC dựa trên IP. Người thiết kế tập trung vào kiến trúc và giao tiếp giữa các khối, còn Vivado đảm nhiệm phần lớn việc sinh kết nối mức thấp.

Bước 2 - Thêm ZYNQ7 Processing System

Cơ sở lý thuyết: ZYNQ7 Processing System là phần PS cứng của Zynq-7000, chứa hai lõi ARM Cortex-A9, DDR controller, clock/reset, interrupt controller và nhiều peripheral. PS có các cổng AXI General Purpose, High Performance và ACP để trao đổi dữ liệu với PL. Trong Lab này, cổng M_AXI_GP0 đóng vai trò AXI master do ARM điều khiển để truy cập AXI GPIO nằm trong PL.

Mục đích của bước: Đưa bộ xử lý ARM và hạ tầng PS vào hệ thống để sau này chương trình C có thể điều khiển phần cứng PL thông qua giao dịch AXI memory-mapped.

Thao tác thực hiện: Trong Block Design, nhấn Add IP, tìm ZYNQ7 Processing System và thêm vào sơ đồ. Vivado tạo instance processing_system7_0. Đây là phần PS của chip Zynq, chứa ARM Cortex-A9 và các tài nguyên hệ thống cố định.



Hình 3. ZYNQ7 Processing System được thêm vào Block Design.

Kết quả và ý nghĩa: Khối processing_system7_0 là trung tâm điều khiển của hệ thống. Ứng dụng C ở Exercise 1C sẽ chạy trên Cortex-A9 trong khối này, còn AXI GPIO sẽ nằm trong PL. Vì vậy PS cần mở cổng AXI master để truy cập ngoại vi ở PL.

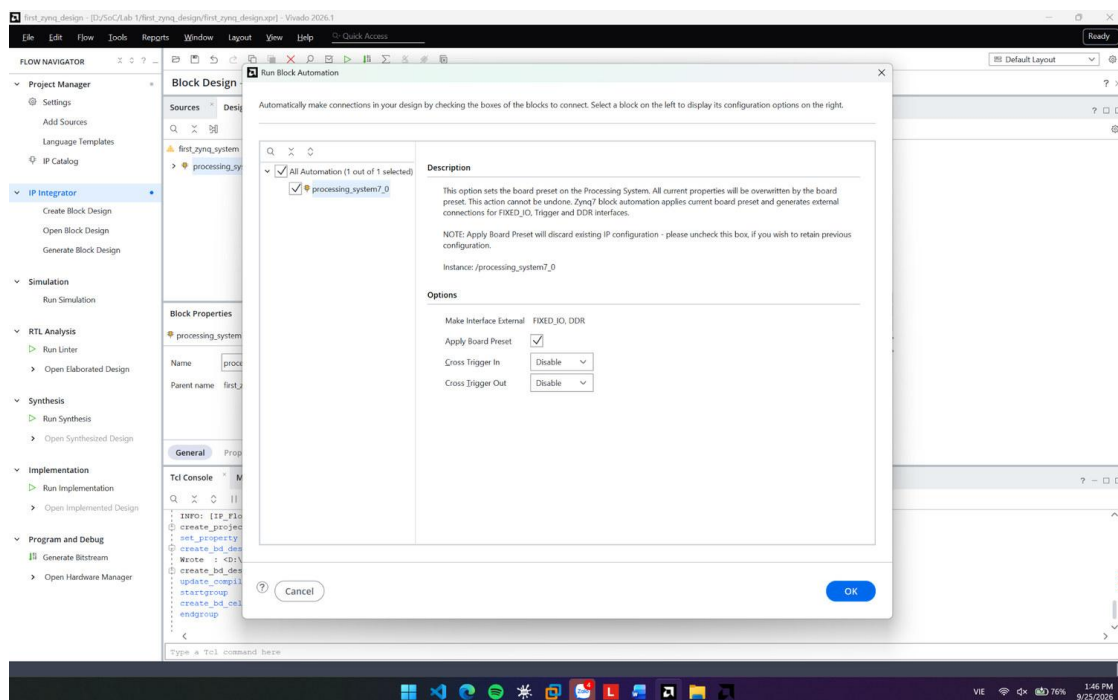
Kiến thức rút ra: Qua bước này có thể thấy Zynq không chỉ là FPGA mà là một SoC kết hợp CPU và logic lập trình được. Phần mềm chạy trên ARM không điều khiển chân LED trực tiếp; nó truy cập ngoại vi PL thông qua một đường bus được ánh xạ địa chỉ.

Bước 3 - Run Block Automation cho Processing System

Cơ sở lý thuyết: PS cần rất nhiều cấu hình board-specific như DDR timing, MIO, clock và FIXED_IO. Board Automation sử dụng preset của board để áp các tham số đã được nhà sản xuất xác định, nhờ đó tránh phải cấu hình thủ công các interface phức tạp. DDR và FIXED_IO là các kết nối thiết yếu để PS hoạt động với bộ nhớ và các chân hệ thống cố định.

Mục đích của bước: Cấu hình nhanh PS theo ZC706 và đưa các interface DDR/FIXED_IO ra ngoài Block Design để hệ thống có nền PS hợp lệ trước khi nối thêm peripheral trong PL.

Thao tác thực hiện: Nhấn Run Block Automation. Giữ lựa chọn Apply Board Preset và cho phép Vivado đưa DDR cùng FIXED_IO ra ngoài Block Design. Nhấn OK/Run để công cụ cấu hình PS theo board preset ZC706.



Hình 4. Cửa sổ Run Block Automation cho Zynq Processing System.

Kiến thức rút ra: Bước này cho thấy preset board là cầu nối giữa kiến trúc logic và phần cứng thực tế. Không phải mọi cấu hình PS đều nên nhập thủ công; với board chuẩn, automation giúp giảm lỗi và đảm bảo tham số DDR/MIO phù hợp.

Cơ sở lý thuyết: AXI GPIO là một AXI slave memory-mapped. CPU ghi/đọc các thanh ghi của AXI GPIO bằng địa chỉ, trong khi AXI SmartConnect làm nhiệm vụ định tuyến giao dịch từ AXI master của PS tới slave tương ứng. AXI4-Lite thường được dùng cho peripheral điều khiển vì đơn giản, không cần burst và phù hợp với truy cập thanh ghi. Tutorial cũ dùng AXI Interconnect; Vivado 2026.1 sử dụng SmartConnect với mục tiêu chức năng tương đương trong hệ thống này.

Thao tác thực hiện: Nhấn Add IP và thêm AXI GPIO. Sau đó chọn Run Connection Automation, đánh dấu cổng /axi_gpio_0/S_AXI và kết nối tới /processing_system7_0/M_AXI_GP0. Trên Vivado 2026.1, công cụ tự tạo axi_smc (AXI SmartConnect), khởi reset rst ps7_0_50M và nối clock/reset thích hợp.



Kết quả và ý nghĩa: Đường M_AXI_GP0 → axi_smc → axi_gpio_0/S_AXI tạo đường truyền memory-mapped từ ARM Cortex-A9 tới ngoại vi trong PL. SmartConnect thực hiện chức năng interconnect mà tutorial cũ mô tả bằng AXI Interconnect. Processor System Reset bảo đảm các khối AXI được reset đồng bộ.

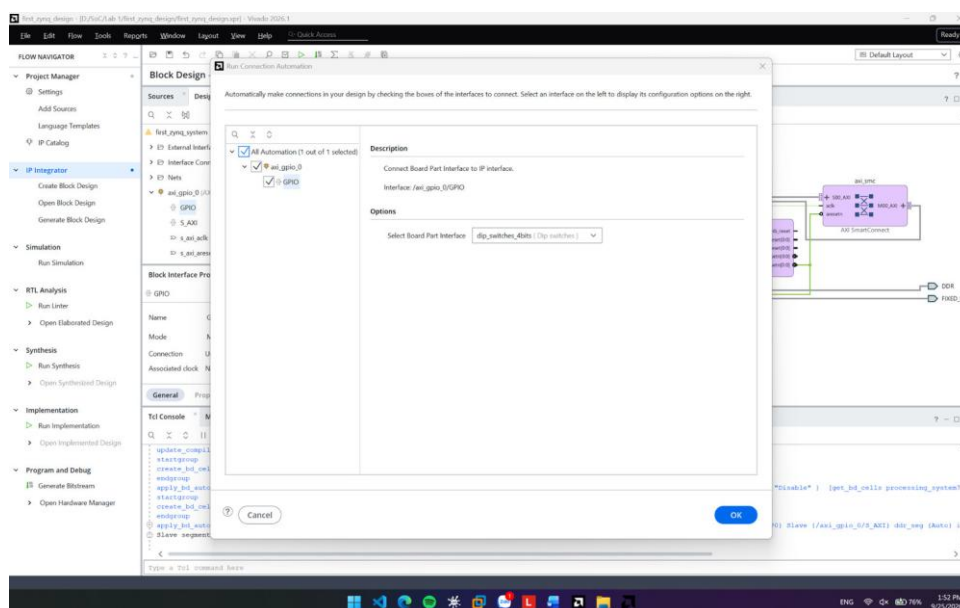
Kiến thức rút ra: Qua bước này có thể hiểu bản chất của hardware/software codesign: một câu lệnh C cuối cùng trở thành transaction trên bus AXI. SmartConnect không phải nơi xử lý thuật toán LED mà là hạ tầng liên kết và định tuyến dữ liệu giữa master và slave.

Bước 5 - Kết nối GPIO ra LED của board

Cơ sở lý thuyết: AXI GPIO chỉ tạo ra tín hiệu logic trong PL; để tín hiệu có ý nghĩa ở cấp board, nó phải được nối tới cổng ngoài tương ứng. Interface led_4bits của ZC706 đại diện cho bốn đường tín hiệu LED trong board definition. Việc nối channel GPIO ra interface này tạo liên kết từ thanh ghi AXI GPIO đến các net LED vật lý/logic quan sát được.

Mục đích của bước: Hoàn tất đường dữ liệu từ phần mềm trên PS tới ngõ ra LED trong PL, biến giá trị ghi vào AXI GPIO thành trạng thái bốn bit led 4bits[3:0].

Thao tác thực hiện: Chọn interface GPIO của axi_gpio_0 và dùng Connection Automation/Make External để nối với interface led_4bits của ZC706. Tutorial ZedBoard sử dụng LEDs_8Bits; do board ZC706 có interface LED 4-bit trong board definition, bài này sử dụng 4 bit.

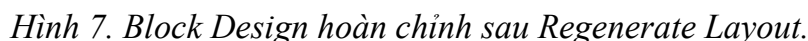


Hình 6. Kết nối AXI GPIO tới interface led 4bits của ZC706.

Kiểm thức rút ra: Bước này giúp nhận ra rằng peripheral AXI và chân I/O là hai lớp khác nhau. CPU chỉ ghi peripheral; chính thiết kế PL mới đưa dữ liệu peripheral ra các chân hoặc interface của board.

Cơ sở lý thuyết: Regenerate Layout chỉ sắp xếp lại vị trí hiển thị của các block và đường nối trong canvas, không làm thay đổi netlist hay chức năng logic. Tuy nhiên, trong thiết kế SoC, sơ đồ rõ ràng giúp kiểm tra nhanh cấu trúc master-slave, clock, reset và external interface.

Thao tác thực hiện: Nhấn Regenerate Layout để Vivado sắp xếp lại sơ đồ sau khi các kết nối đã hoàn tất. Thao tác này không thay đổi chức năng mạch mà chỉ giúp kiểm tra kiến trúc rõ ràng hơn.



Kết quả và ý nghĩa: Sơ đồ hoàn chỉnh cho thấy bốn khối chính: processing_system7_0, axi_smc, rst_ps7_0_50M và axi_gpio_0. Ba interface ngoài là DDR, FIXED_IO và led_4bits. Từ hình có thể lần theo đường điều khiển từ ARM PS qua AXI tới GPIO ở PL.

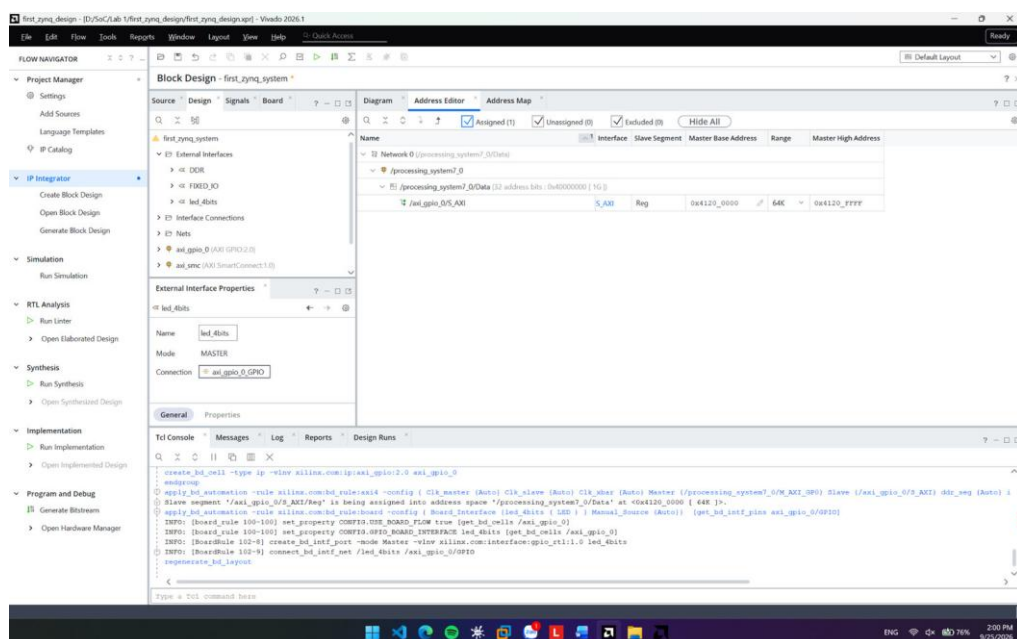
Kiến thức rút ra: Qua bước này có thể rèn kỹ năng đọc kiến trúc theo đường tín hiệu: PS → SmartConnect → AXI GPIO → LED, đồng thời nhận biết reset và clock là các đường hạ tầng đi kèm.

Bước 7 - Kiểm tra Address Editor

Cơ sở lý thuyết: Trong hệ thống memory-mapped, mỗi AXI slave được gán một vùng địa chỉ trong không gian địa chỉ của processor. Khi ARM truy cập một địa chỉ thuộc vùng đó, interconnect giải mã địa chỉ và chuyển transaction tới đúng slave. Vì vậy base address là thông tin liên kết trực tiếp giữa phần cứng và phần mềm; BSP sẽ sinh macro từ memory map để driver sử dụng.

Mục đích của bước: Xác nhận AXI GPIO đã được ánh xạ đúng tại 0x41200000 và không xung đột với vùng địa chỉ khác, từ đó phần mềm có thể truy cập peripheral một cách xác định.

Thao tác thực hiện: Mở Address Editor. Kiểm tra segment của axi_gpio_0/S_AXI trong không gian địa chỉ của processing_system7_0. Bài thực hành nhận được Base Address 0x41200000, Range 64K và High Address 0x4120FFFF.



Hình 8. Address Editor và vùng địa chỉ của AXI GPIO.

Kết quả và ý nghĩa: Address Editor xác định cách phần mềm trên PS nhìn thấy ngoại vi PL. Khi CPU truy cập địa chỉ bắt đầu từ 0x41200000, giao dịch AXI được SmartConnect định tuyến tới AXI GPIO. Thông tin này được đưa vào hardware platform và sinh macro trong xparameters.h cho phần mềm.

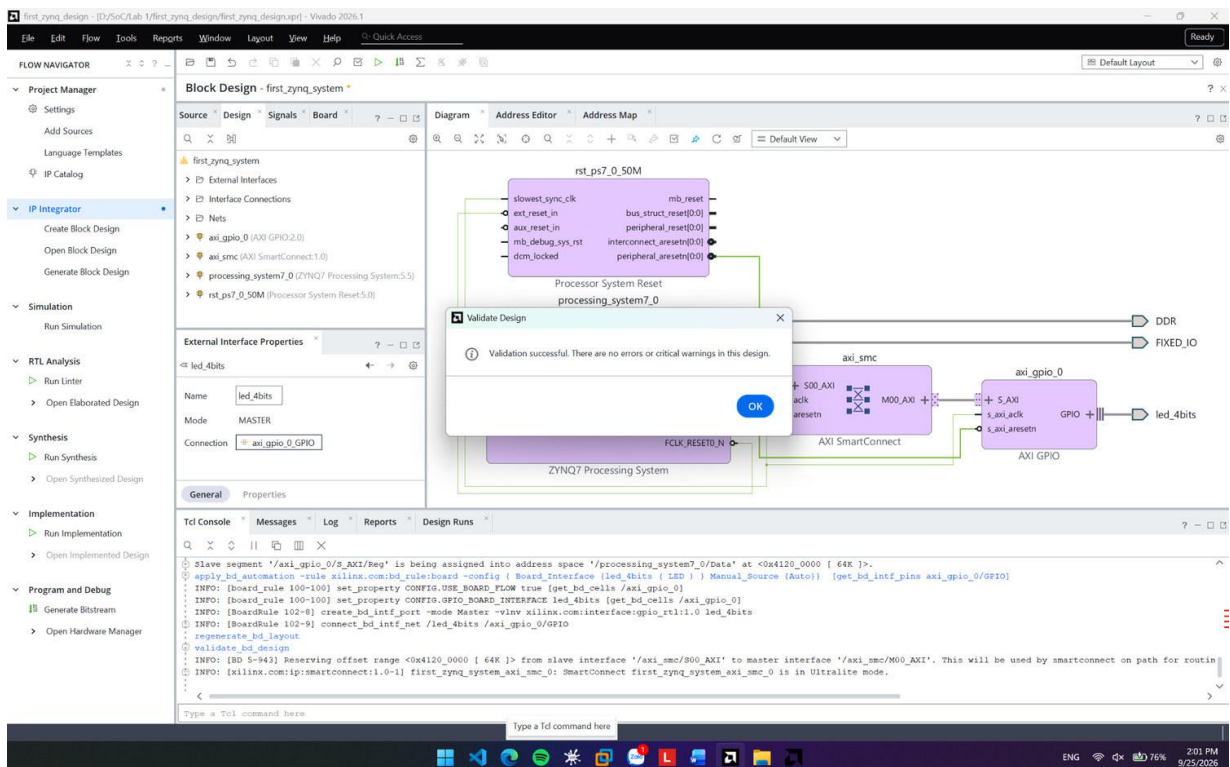
Kiến thức rút ra: Bước này giúp hiểu khái niệm memory-mapped I/O: peripheral được CPU nhìn như một vùng nhớ. Địa chỉ trong xparameters.h không phải con số tùy ý mà xuất phát trực tiếp từ Address Editor của thiết kế hardware.

Bước 8 - Validate Design

Cơ sở lý thuyết: Validate Design là kiểm tra tính nhất quán ở mức Block Design trước khi sinh HDL. Vivado rà soát interface, clock/reset, tham số IP, địa chỉ và các rule kết nối. Đây là bước phát hiện lỗi kiến trúc sớm, trước khi lỗi lan sang synthesis hoặc implementation vốn mất nhiều thời gian hơn.

Mục đích của bước: Xác nhận hệ thống đủ điều kiện để tạo HDL wrapper và chuyển sang luồng RTL/synthesis.

Thao tác thực hiện: Chọn Tools → Validate Design để Vivado kiểm tra các rule của Block Design: interface, clock, reset, địa chỉ và kết nối IP. Chỉ tiếp tục sinh HDL khi validation thành công.



Hình 9. Kết quả Validate Design thành công.

Kết quả và ý nghĩa: Thông báo validation successful cho biết Block Design không có lỗi kết nối nghiêm trọng. Đây là checkpoint trước Create HDL Wrapper; nếu còn lỗi ở clock/reset hoặc interface AXI, synthesis có thể thất bại hoặc hệ thống hoạt động không đúng.

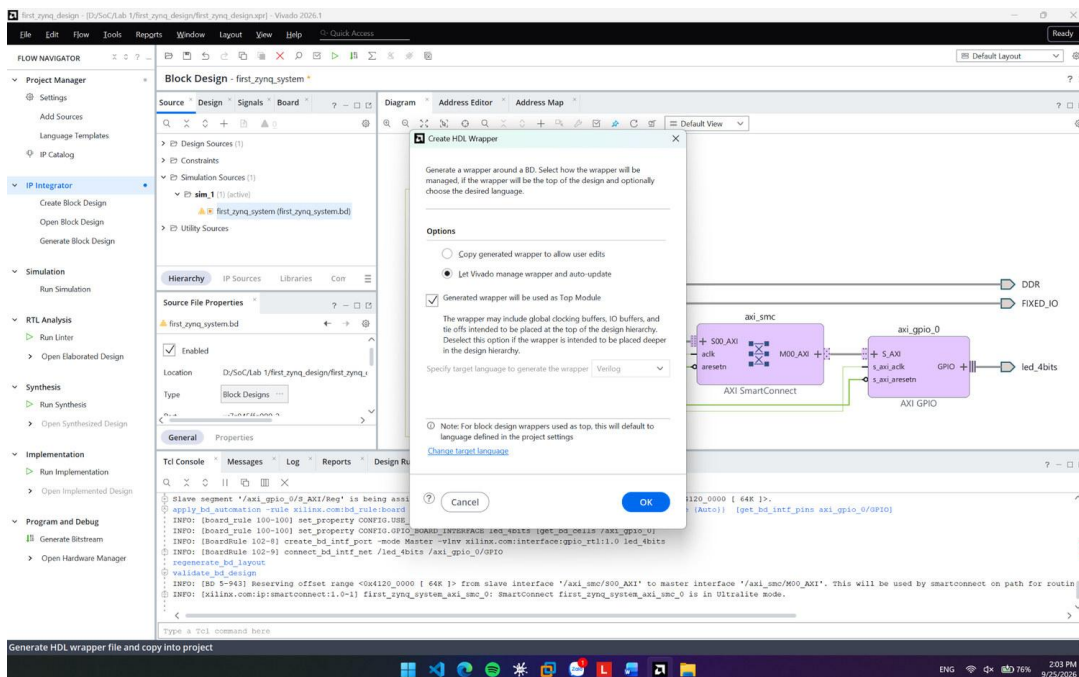
Kiến thức rút ra: Qua bước này học được nguyên tắc thiết kế số quan trọng: luôn kiểm tra tính hợp lệ ở mức cao trước khi chạy các bước tốn thời gian. Lỗi clock/reset hoặc AXI nên được xử lý ở Block Design thay vì chờ tới implementation.

Bước 9 - Create HDL Wrapper

Cơ sở lý thuyết: Block Design là mô tả đồ họa, trong khi synthesis cần một top-level HDL module. HDL Wrapper tạo lớp Verilog bao quanh Block Design và đưa các external interface như DDR, FIXED_IO, LED thành port của top module. Khi để Vivado manage wrapper, file wrapper sẽ tự cập nhật khi Block Design thay đổi.

Mục đích của bước: Chuyển kiến trúc IP Integrator sang dạng top-level Verilog mà các công cụ RTL Analysis, Synthesis và Implementation có thể xử lý.

Thao tác thực hiện: Trong Sources, nhấp phải Block Design và chọn Create HDL Wrapper. Chọn Let Vivado manage wrapper and auto-update. Do project đặt Target language là Verilog nên wrapper sinh ra là first_zynq_system_wrapper.v và được dùng làm top-level design.



Hình 10. Tạo HDL Wrapper cho Block Design.

Kết quả và ý nghĩa: Wrapper biến Block Design thành một top module mà luồng RTL/Synthesis/Implementation có thể xử lý. Tùy chọn Vivado manage wrapper giúp wrapper tự cập nhật khi Block Design thay đổi, tránh phải sửa tay file top.

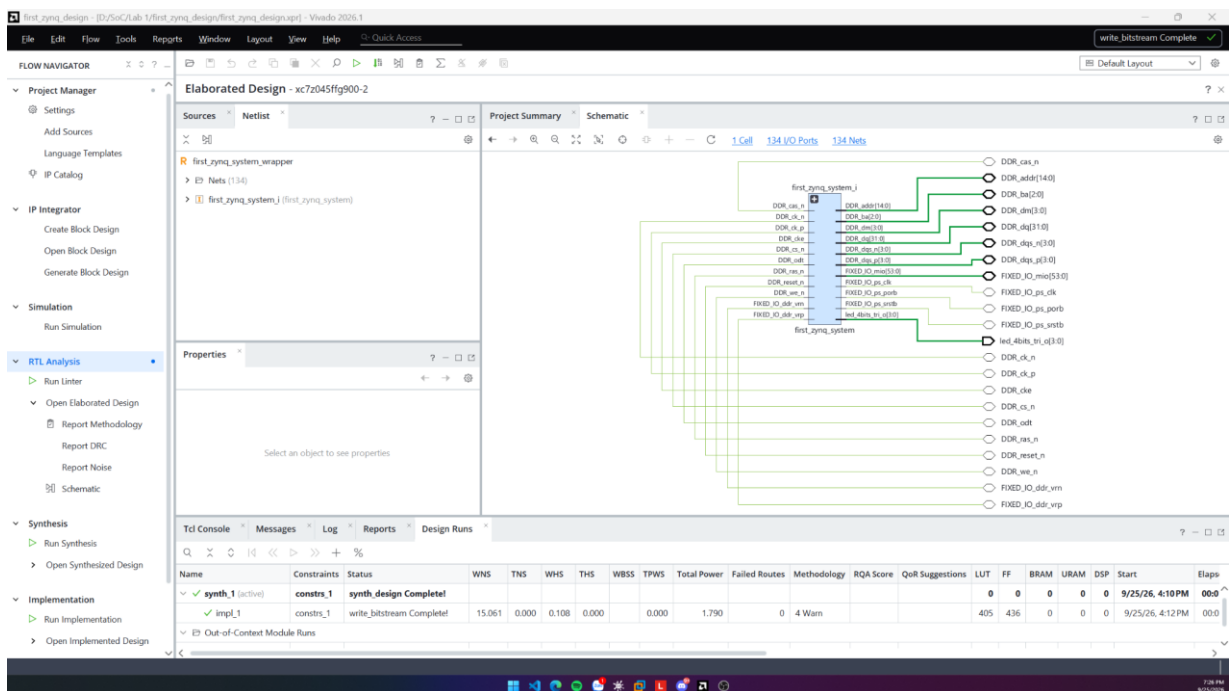
Kiến thức rút ra: Bước này cho thấy công cụ đồ họa cuối cùng vẫn phải được biểu diễn bằng HDL/netlist để đi qua flow FPGA. Wrapper là ranh giới giữa thiết kế IP Integrator và luồng RTL truyền thống.

Bước 10 - Kiểm tra RTL/Elaborated Design

Cơ sở lý thuyết: Elaboration phân tích hierarchy, tham số, generate statement và kết nối để tạo mô hình logic đã được hiểu đầy đủ trước synthesis. Nó chưa ánh xạ sang LUT/FF cụ thể nhưng cho phép kiểm tra top-level ports và cấu trúc thiết kế sau khi Vivado đã giải nghĩa HDL/IP.

Mục đích của bước: Xác nhận wrapper, hierarchy và các cổng ngoài của hệ thống được Vivado nhận đúng trước khi tổng hợp xuống công nghệ FPGA.

Thao tác thực hiện: Trong Flow Navigator, mở RTL Analysis → Open Elaborated Design và xem Schematic. Đây là bước kiểm tra cấu trúc logic sau elaboration, trước khi ánh xạ xuống tài nguyên FPGA.



Hình 11. Sơ đồ Elaborated Design của first_zynq_system_wrapper.

Kết quả và ý nghĩa: Hình RTL cho thấy top wrapper với các cổng DDR, FIXED_IO và led_4bits[3:0]. Ở Design Runs phía dưới, synth_1 và impl_1 đã được tạo; đây là bằng

chúng thiết kế đã được Vivado nhận dạng đúng top-level và các cổng I/O cần thiết xuất hiện trong netlist elaborated.

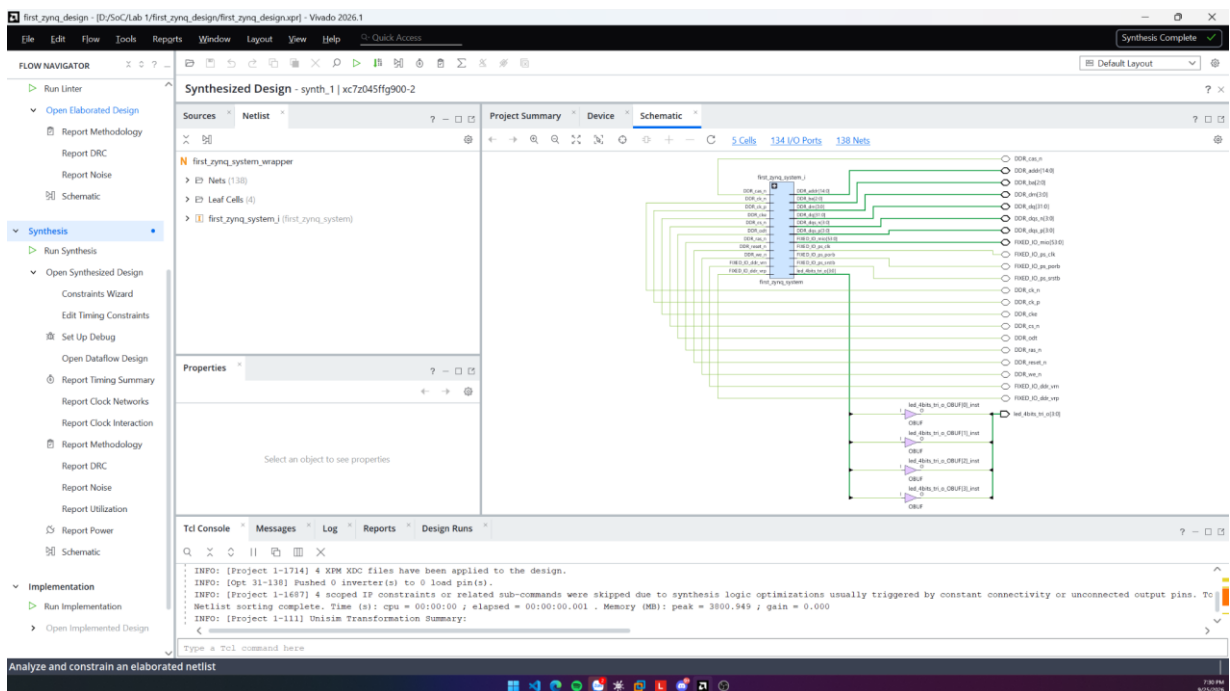
Kiến thức rút ra: Qua bước này có thể phân biệt elaboration với synthesis: elaboration kiểm tra và triển khai cấu trúc RTL về mặt logic, còn synthesis mới tối ưu và ánh xạ thiết kế sang tài nguyên phần cứng.

Bước 11 - Run Synthesis

Cơ sở lý thuyết: Synthesis chuyển RTL và netlist IP thành netlist công nghệ gồm LUT, flip-flop, buffer, block RAM và các primitive của FPGA. Công cụ đồng thời tối ưu logic dựa trên ràng buộc và cấu trúc thiết kế. Kết quả synthesis vẫn chưa có vị trí vật lý cuối cùng trên chip.

Mục đích của bước: Biến mô tả logic của hệ thống thành phần cứng khả thi trên Zynq-7000 và kiểm tra xem các kết nối LED, AXI và logic hạ tầng có được tổng hợp thành công hay không.

Thao tác thực hiện: Chọn Run Synthesis. Synthesis chuyển mô tả RTL/IP thành netlist công nghệ phù hợp FPGA Zynq-7000. Sau khi hoàn tất, chọn Open Synthesized Design → Schematic để kiểm tra netlist sau tổng hợp.



Hình 12. Synthesized Design sau khi Run Synthesis.

Kết quả và ý nghĩa: Góc trên bên phải hiển thị Synthesis Complete. Sơ đồ đã có các buffer ngõ ra OBUF cho led_4bits, cho thấy các tín hiệu LED đã được đưa tới I/O logic.

Synthesis là bước biến kiến trúc IP ở mức cao thành tập phân tử logic mà Implementation có thể place và route.

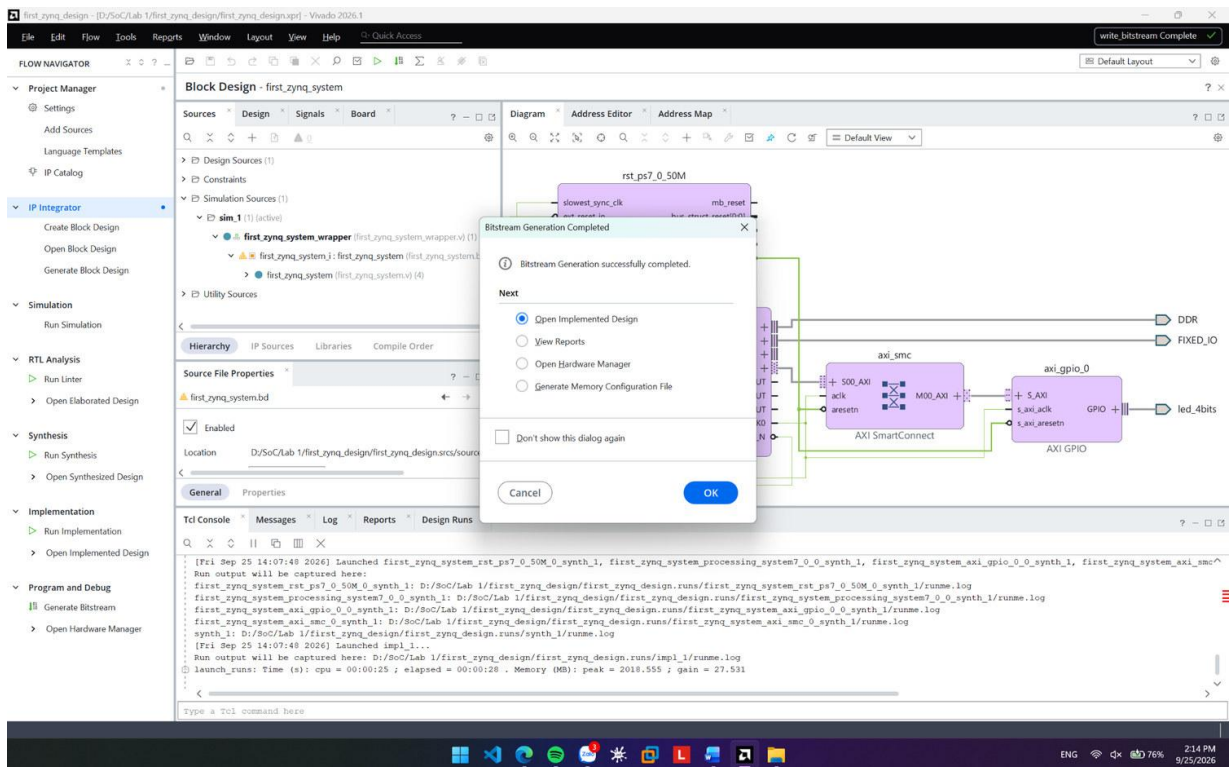
Kiến thức rút ra: Bước này giúp hiểu rằng code hoặc Block Design không trực tiếp trở thành bitstream. Synthesis là giai đoạn chuyển từ mô tả hành vi/cấu trúc sang netlist phân cứng cụ thể.

Bước 12 - Run Implementation và Generate Bitstream

Cơ sở lý thuyết: Implementation gồm các quá trình tối ưu vật lý, placement và routing để gán các primitive đã synthesis vào vị trí thực trên FPGA và nối chúng bằng mạng routing. Generate Bitstream mã hóa cấu hình cuối cùng thành file .bit để lập trình PL. Chỉ khi timing và routing hợp lệ mới có thể xem bitstream là cấu hình phần cứng hoàn chỉnh.

Mục đích của bước: Hoàn thiện thiết kế ở mức vật lý và tạo file cấu hình PL dùng cho export hardware hoặc nạp lên FPGA khi có board thật.

Thao tác thực hiện: Sau synthesis, chạy Implementation. Khi implementation thành công, chọn Generate Bitstream. Vivado thực hiện các bước cuối và ghi file .bit cho PL. Nếu project chưa có implementation result, Vivado sẽ tự chạy synthesis/implementation trước khi write_bitstream.



Hình 13. Thông báo Bitstream Generation Completed.

Kết quả và ý nghĩa: Thông báo này xác nhận write_bitstream đã hoàn tất. File bitstream mô tả cấu hình phần PL, gồm AXI SmartConnect, reset logic, AXI GPIO và kết nối LED. Trong tutorial gốc, file này được nạp xuống ZedBoard; trong bài hiện tại nó được kèm vào XSA để chuẩn bị flow Vitis, không nạp board vật lý.

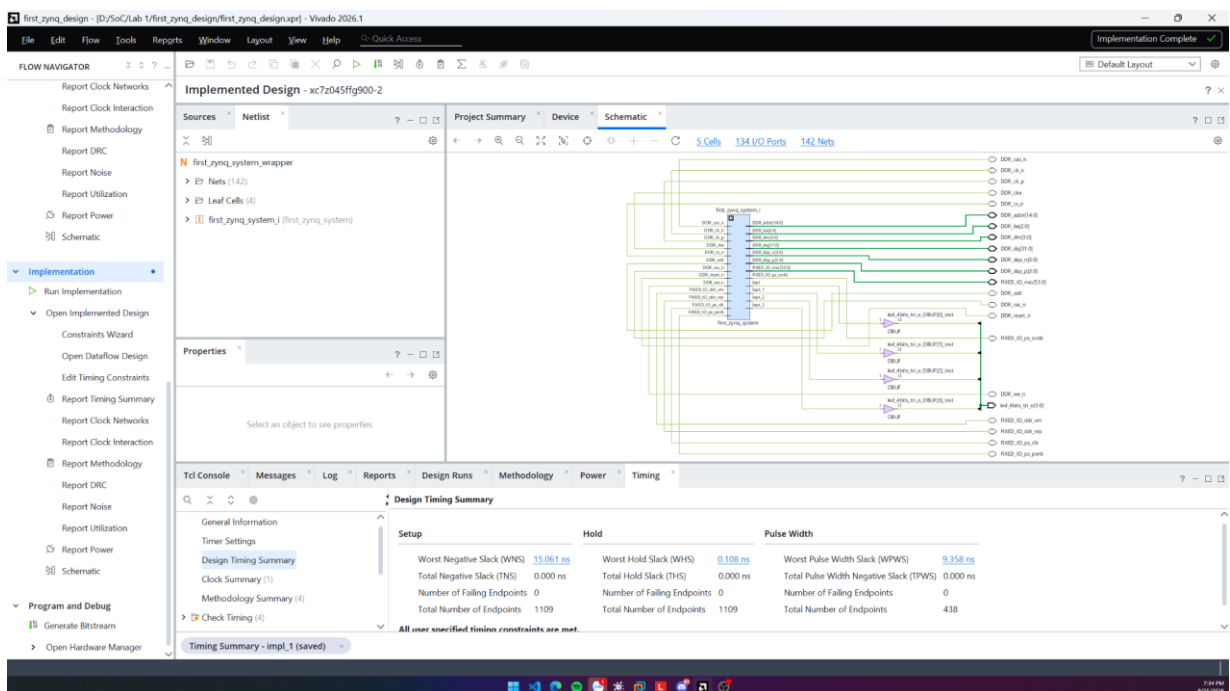
Kiến thức rút ra: Qua bước này học được chuỗi chuyển đổi đầy đủ: kiến trúc → HDL/netlist → synthesis → placement/routing → bitstream. Bitstream là sản phẩm cuối của phần hardware, không phải file source code.

Bước 13 - Kiểm tra Implemented Design và Timing

Cơ sở lý thuyết: Timing analysis kiểm tra các đường truyền dữ liệu theo ràng buộc clock. WNS là Worst Negative Slack của setup, WHS liên quan hold và WPWS liên quan pulse width. Slack dương nghĩa là đường timing có dư thời gian; failing endpoints bằng 0 cho thấy không có endpoint vi phạm ràng buộc tương ứng.

Mục đích của bước: Xác nhận thiết kế sau place-and-route đáp ứng yêu cầu thời gian, tránh trường hợp bitstream được tạo nhưng mạch có nguy cơ hoạt động không ổn định ở tần số yêu cầu.

Thao tác thực hiện: Mở Open Implemented Design và chọn Report Timing Summary. Kiểm tra WNS, WHS, WPWS và số failing endpoints để xác nhận các ràng buộc timing được đáp ứng.



Hình 14. Implemented Design và Design Timing Summary.

Kết quả và ý nghĩa: Kết quả ghi nhận WNS = 15.061 ns, WHS = 0.108 ns và WPWS = 9.358 ns; các mục Number of Failing Endpoints đều bằng 0. Điều này cho thấy các timing constraint do hệ thống sử dụng đã được đáp ứng sau place-and-route. Hình cũng cho thấy netlist implemented với các cổng DDR/FIXED_IO/LED.

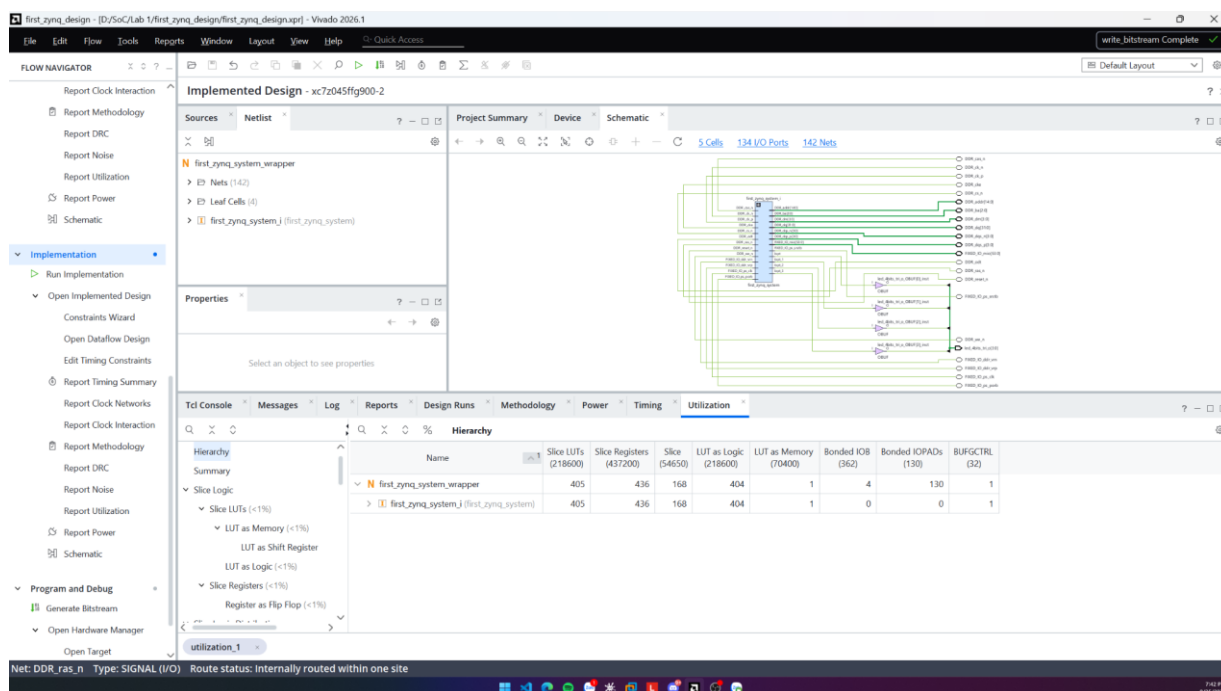
Kiến thức rút ra: Bước này giúp hiểu rằng mạch đúng logic chưa đủ; thiết kế số còn phải đúng timing. Báo cáo timing là tiêu chí chất lượng quan trọng trước khi coi implementation là đạt.

Bước 14 - Kiểm tra Report Utilization

Cơ sở lý thuyết: Utilization cho biết thiết kế tiêu thụ bao nhiêu LUT, register, slice, I/O, clock buffer và các tài nguyên chuyên dụng khác. Tỷ lệ sử dụng ảnh hưởng tới khả năng mở rộng, congestion và đôi khi ảnh hưởng timing. Đối với một hệ thống GPIO đơn giản, mức sử dụng nhỏ là hợp lý.

Mục đích của bước: Đánh giá quy mô tài nguyên phần PL và chứng minh hệ thống còn nhiều dư địa để tích hợp thêm IP trong các bài Lab sau.

Thao tác thực hiện: Trong Implemented Design, mở Report Utilization để quan sát lượng tài nguyên PL được sử dụng. Đây là báo cáo cần thiết để đánh giá quy mô phần cứng sau implementation.



Hình 15. Report Utilization của thiết kế sau Implementation.

Kết quả và ý nghĩa: Báo cáo cho thấy thiết kế sử dụng khoảng 405 Slice LUTs, 436 Slice Registers, 168 Slices, 4 Bonded IOB và 1 BUFGCTRL. Tỷ lệ sử dụng rất nhỏ so với dung lượng xc7z045ffg900-2, phù hợp với hệ thống Lab chỉ gồm AXI GPIO và logic kết nối.

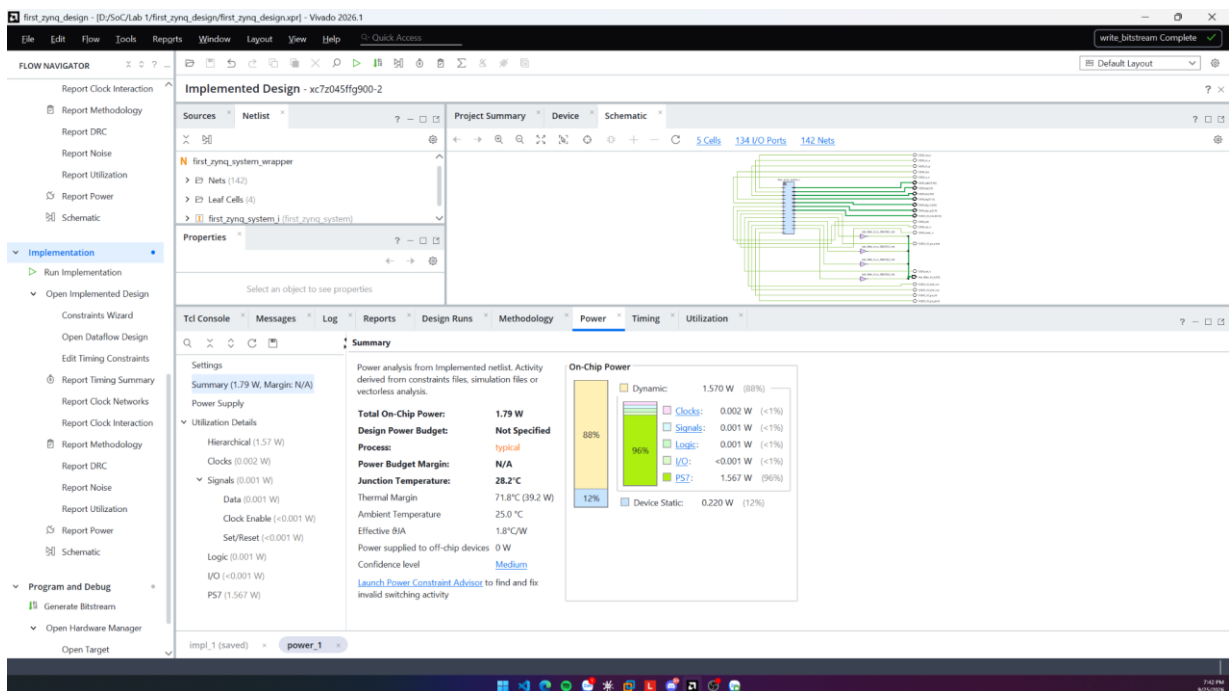
Kiến thức rút ra: Qua bước này học cách đọc chi phí phần cứng của một kiến trúc, thay vì chỉ quan tâm thiết kế có chạy hay không. Đây là nền tảng cho tối ưu diện tích và lựa chọn device.

Bước 15 - Kiểm tra Report Power

Cơ sở lý thuyết: Vivado Power Report ước lượng công suất dynamic và static dựa trên kiến trúc đã implementation, clock và switching activity. Dynamic power phụ thuộc hoạt động chuyển mạch; static power là công suất rò của thiết bị. Nếu activity chỉ là mặc định hoặc ước lượng, kết quả phải được xem là dự báo chứ không phải số đo thực nghiệm.

Mục đích của bước: Đánh giá sơ bộ mức tiêu thụ năng lượng của hệ thống và nhận biết thành phần nào đóng góp lớn vào công suất tổng.

Thao tác thực hiện: Trong Implemented Design, mở Report Power để lấy ước lượng công suất dựa trên netlist đã implementation và activity mặc định/ước lượng của công cụ.



Hình 16. Report Power của hệ thống Zynq.

Kết quả và ý nghĩa: Vivado ước lượng Total On-Chip Power khoảng 1.79 W, trong đó Dynamic khoảng 1.570 W và Device Static khoảng 0.220 W. Phần PS7 chiếm phần

lớn dynamic power. Confidence level hiển thị Medium, vì vậy các con số này là ước lượng của công cụ chứ không phải số đo công suất thực tế trên board.

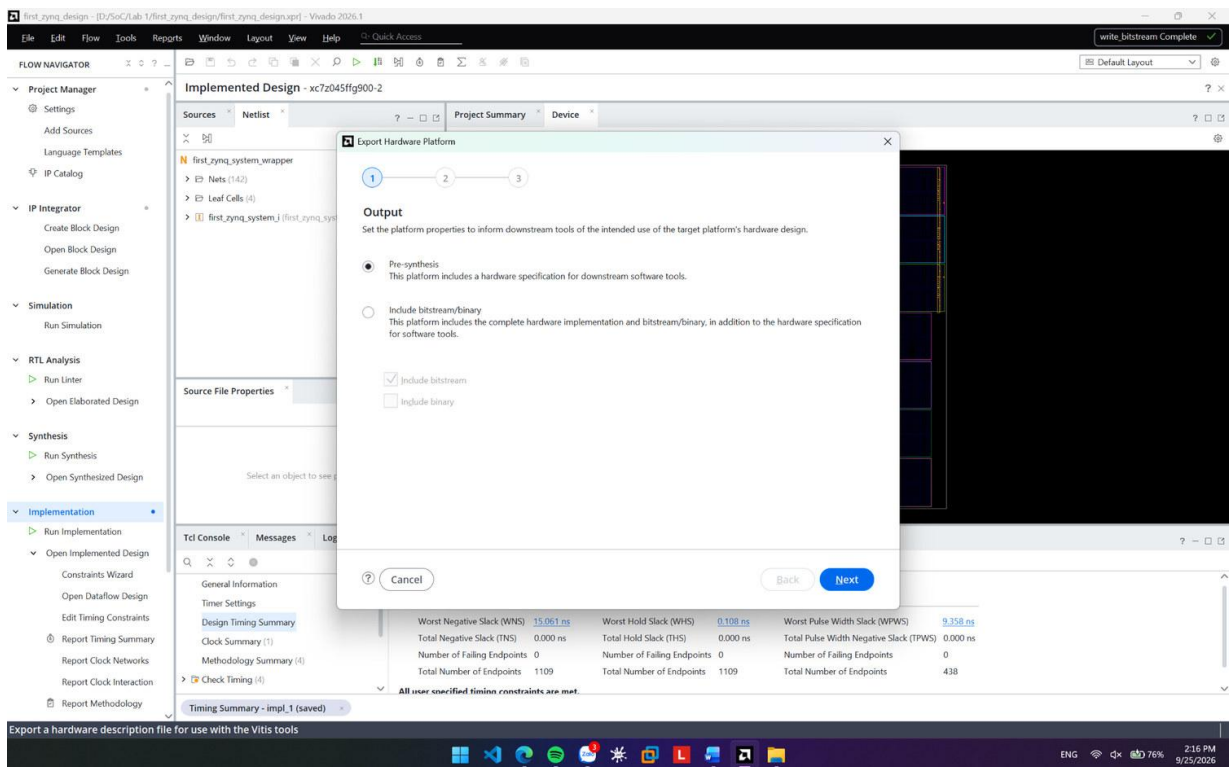
Kiến thức rút ra: Bước này giúp liên hệ thiết kế logic với khía cạnh năng lượng. Trong SoC, PS có thể chiếm phần lớn dynamic power dù phần PL của bài Lab rất nhỏ; do đó tối ưu công suất phải xét toàn hệ thống chứ không chỉ LUT/FF.

Bước 16 - Export Hardware Platform

Cơ sở lý thuyết: Hardware handoff là bước chuyển thông tin từ nhóm thiết kế phần cứng sang môi trường phát triển phần mềm. File XSA chứa mô tả processor, memory map, peripheral, clock, interrupt và có thể kèm bitstream. Vitis dùng thông tin này để sinh platform, BSP, driver và macro phần cứng cho application.

Mục đích của bước: Đóng gói thiết kế Vivado thành một hardware platform mà Vitis có thể hiểu và sử dụng để phát triển phần mềm cho đúng kiến trúc đã implementation.

Thao tác thực hiện: Chọn File → Export → Export Hardware. Ở flow Vivado 2026.1, chọn export hardware platform và bật Include bitstream/binary để bitstream được đóng gói cùng mô tả phần cứng.



Hình 17. Tùy chọn Export Hardware Platform có kèm bitstream.

Kết quả và ý nghĩa: Việc include bitstream tương đương ý nghĩa bước Export Hardware for SDK trong tutorial cũ: môi trường phần mềm nhận được mô tả processor, memory map, peripheral và file cấu hình PL. Đây là cầu nối chính giữa Exercise 1B và Exercise 1C.

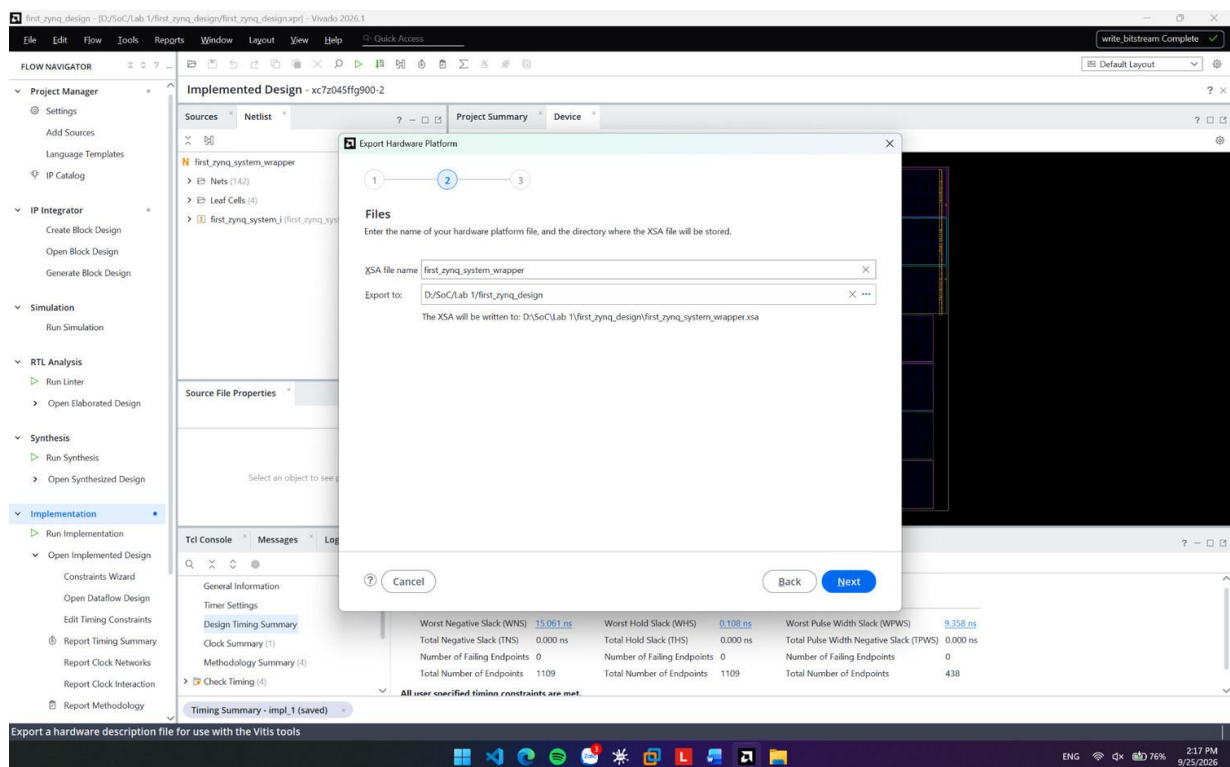
Kiến thức rút ra: Bước này minh họa rõ ranh giới giữa hardware và software trong codesign. Phần mềm không tự biết AXI GPIO nằm ở đâu; thông tin đó được truyền từ Vivado sang Vitis thông qua XSA.

Bước 17 - Lưu file XSA

Cơ sở lý thuyết: XSA là định dạng handoff hiện đại thay cho flow Export Hardware for SDK của các phiên bản Vivado cũ. Nó cho phép tái sử dụng một hardware design để tạo nhiều software domain/application khác nhau mà không cần thiết kế lại PL.

Mục đích của bước: Tạo artifact phần cứng ổn định làm đầu vào chính cho Exercise 1C trong Vitis.

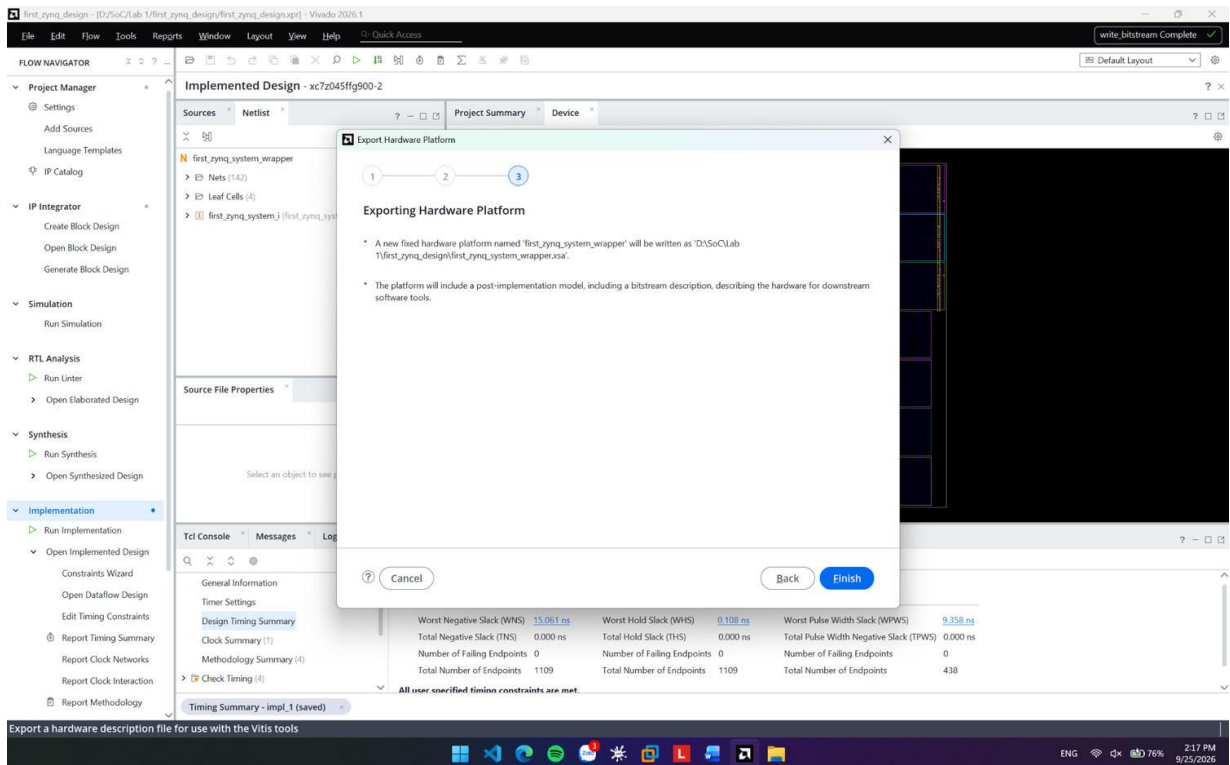
Thao tác thực hiện: Đặt tên hardware platform là first_zynq_system_wrapper.xsa và lưu trong thư mục project. XSA là định dạng hardware handoff của flow Vitis hiện đại.



Hình 18. Chọn tên và vị trí file first_zynq_system_wrapper.xsa.

Kết quả và ý nghĩa: File XSA chứa thông tin cần thiết để Vitis tạo platform và BSP. Từ XSA, Vitis biết processor ps7_cortexa9_0, các driver tương ứng và memory map của AXI GPIO, bao gồm địa chỉ 0x41200000.

Kiến thức rút ra: Qua bước này học được cách tách vòng đời phần cứng và phần mềm: sau khi XSA ổn định, phát triển application có thể tiếp tục độc lập miễn là memory map và interface hardware không thay đổi.



Hình 19. Summary của quá trình Export Hardware Platform.

Trang Summary là bước kiểm tra cuối trước khi export. Sau khi hoàn tất, phần phần cứng của Lab đã sẵn sàng cho phát triển phần mềm. Đây là điểm tương đương với việc tutorial mở SDK sau Exercise 1B.

Exercise 1C - Tạo ứng dụng phần mềm điều khiển LED trong Vitis

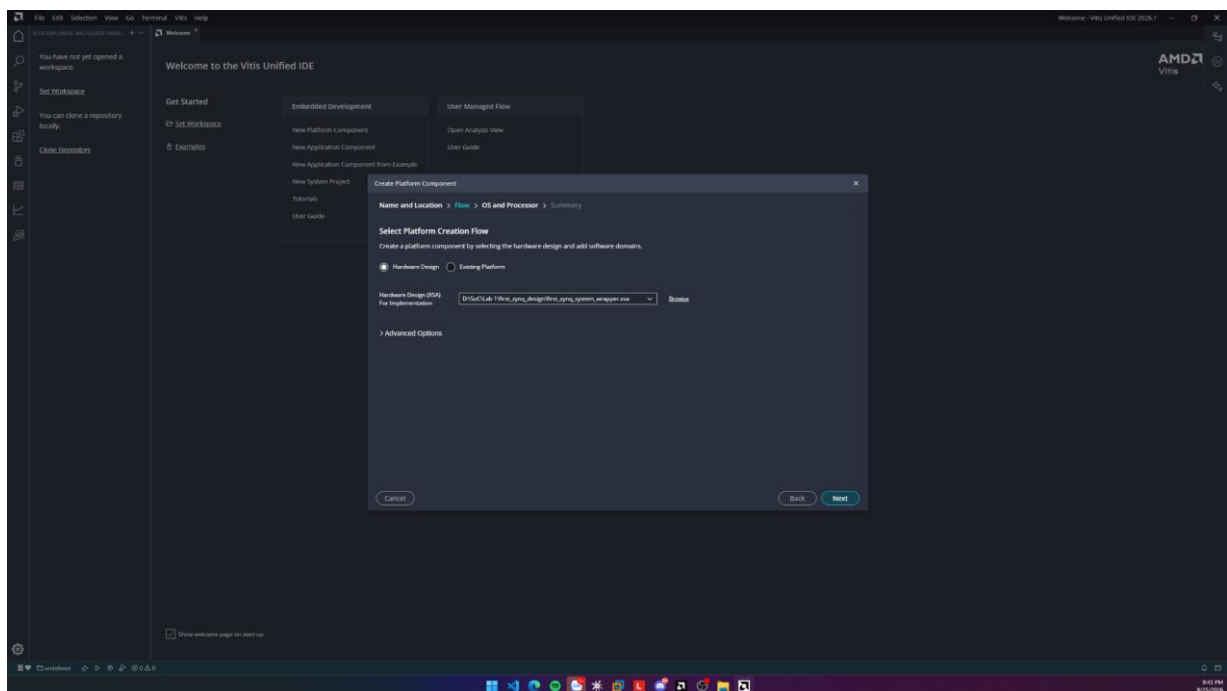
Theo Tutorial 1, mục tiêu Exercise 1C là tạo một software application chạy trên Zynq Processing System và giao tiếp với AXI GPIO trong PL. Tutorial 2014.1 tạo project LED_test trong SDK, import file C mẫu, build, Program FPGA và Launch on Hardware. Bài hiện tại thực hiện cùng mục tiêu bằng Vitis 2026.1: tạo platform từ XSA, tạo Empty Application, viết main.c, build thành ELF và kiểm chứng behavior bằng mô phỏng do không có board vật lý.

Bước 1 - Tạo Platform Component từ XSA

Cơ sở lý thuyết: Trong Vitis, Platform Component mô tả nền tảng phần cứng và phần mềm cơ sở cho application. Nó được tạo từ XSA để Vitis biết processor, peripheral, address map và các driver có thể sử dụng. Platform không phải application; nó là lớp nền mà một hoặc nhiều application dựa lên.

Mục đích của bước: Chuyển hardware handoff từ Vivado thành môi trường phát triển phần mềm đúng với Zynq PS và AXI GPIO đã thiết kế.

Thao tác thực hiện: Mở Vitis Embedded Development và chọn workspace không chứa khoảng trắng: D:\SoC\vitis_embedded_lab1. Chọn tạo Platform Component theo Hardware Design và trở tới file first_zynq_system_wrapper.xsa đã export từ Vivado. Platform được đặt tên first_zynq_platform.



Hình 20. Chọn hardware design XSA để tạo Platform Component.

Kết quả và ý nghĩa: Đây là bước thay thế cho SDK workspace/hardware platform của Figure 1.22 trong tutorial. Vitis đọc XSA để tạo software platform phù hợp với phần cứng đã thiết kế. Không nên chọn một XSA khác vì driver và memory map có thể không khớp AXI GPIO của project hiện tại.

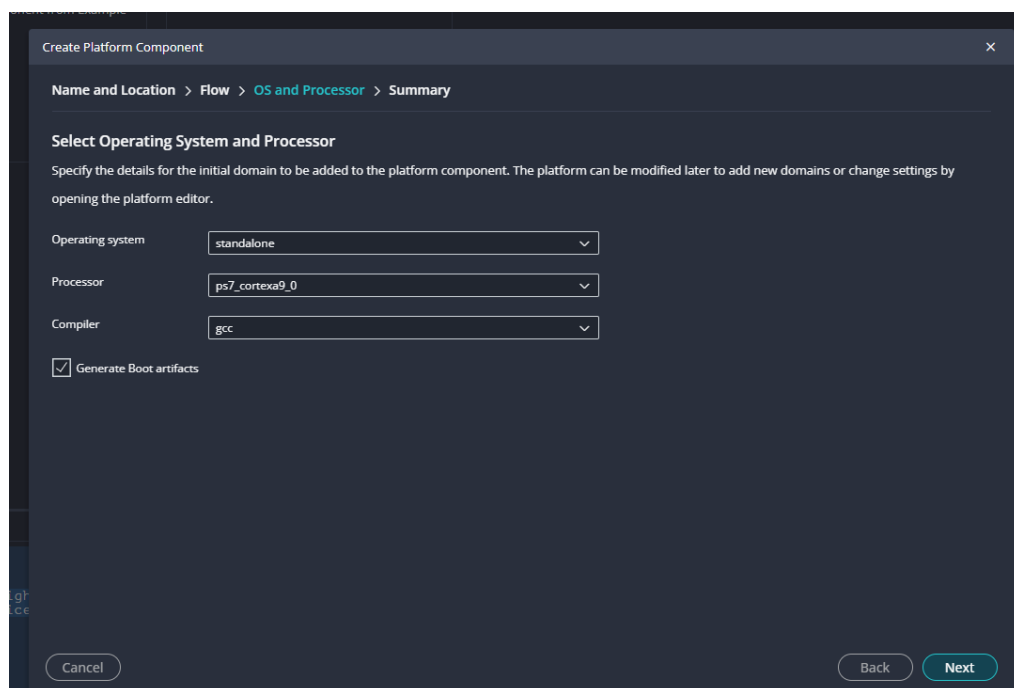
Kiến thức rút ra: Bước này giúp phân biệt ba khái niệm: hardware platform, BSP/domain và application. Việc tách các lớp này làm flow phần mềm có cấu trúc và dễ tái sử dụng hơn.

Bước 2 - Chọn operating system và processor

Cơ sở lý thuyết: Standalone domain là môi trường bare-metal: chương trình chạy trực tiếp trên processor không có hệ điều hành đa nhiệm đầy đủ. BSP cung cấp startup code, thư viện cơ bản và driver peripheral. Chọn ps7_cortexa9_0 xác định chính xác lõi CPU mà application sẽ được compile/link cho nó.

Mục đích của bước: Tạo domain phần mềm phù hợp với mục tiêu Lab: một chương trình C đơn giản chạy trực tiếp trên Cortex-A9 và điều khiển AXI GPIO.

Thao tác thực hiện: Ở trang OS and Processor, chọn Operating system = standalone, Processor = ps7_cortexa9_0, Compiler = gcc và giữ Generate Boot artifacts. Đây là domain bare-metal chạy trực tiếp trên một lõi Cortex-A9, phù hợp với ví dụ SDK của tutorial.



Hình 21. Chọn standalone và processor ps7_cortexa9_0 cho platform.

Kết quả và ý nghĩa: Lựa chọn standalone tạo BSP không có hệ điều hành đầy đủ; application sẽ gọi trực tiếp driver Xilinx/AMD. ps7_cortexa9_0 là processor mục tiêu nơi chương trình led_control_app.elf được link và chạy. Platform Build sau đó sinh BSP và driver AXI GPIO cần thiết.

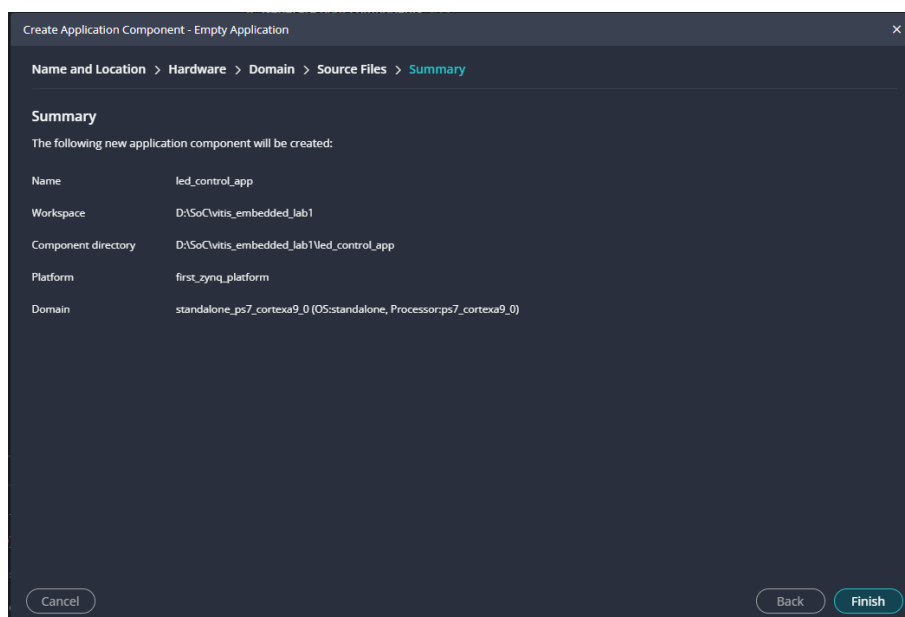
Kiến thức rút ra: Qua bước này hiểu rằng cùng một hardware platform có thể hỗ trợ nhiều software environment khác nhau như standalone, FreeRTOS hoặc Linux. Lựa chọn OS/domain quyết định thư viện, driver và cách application tương tác với phần cứng.

Bước 3 - Tạo Empty Application

Cơ sở lý thuyết: Application Component chứa source code người dùng và được build cho một platform/domain cụ thể. Chọn Empty Application giúp quan sát rõ các bước tự viết main.c và dùng driver XGpio thay vì dựa vào template có sẵn. Đây là lựa chọn tương đương tinh thần tutorial khi tạo một project phần mềm riêng rồi đưa source điều khiển LED vào.

Mục đích của bước: Tạo project phần mềm độc lập gắn đúng với first_zynq_platform và standalone_ps7_cortexa9_0 để chuẩn bị viết chương trình điều khiển LED.

Thao tác thực hiện: Tạo Application Component dạng Empty Application. Đặt tên led_control_app, chọn Platform = first_zynq_platform và Domain = standalone_ps7_cortexa9_0. Không import source có sẵn như tutorial; thay vào đó tạo file main.c trực tiếp trong thư mục src.



Hình 22. Summary khi tạo Empty Application led_control_app.

Kết quả và ý nghĩa: Summary cho thấy application được gắn đúng với first_zynq_platform và domain standalone của ps7_cortexa9_0. Đây là bước tương đương Figure 1.23-1.24 của tutorial, chỉ khác giao diện và cách tổ chức component của Vitis Unified IDE 2026.1.

Kiến thức rút ra: Bước này giúp thấy application luôn phụ thuộc vào platform/domain. Một file C giống nhau có thể không build hoặc chạy đúng nếu chọn sai processor, BSP hoặc hardware handoff.

Bước 4 - Kiểm tra tham số AXI GPIO và viết main.c

Cơ sở lý thuyết: xparameters.h là file được BSP sinh tự động từ hardware platform, chứa địa chỉ và tham số của các peripheral. Driver XGpio cung cấp API mức cao để khởi tạo, cấu hình hướng dữ liệu và ghi/đọc GPIO mà không cần tự thao tác trực tiếp từng thanh ghi. Trong GPIO direction mask của Xilinx, bit 0 nghĩa là output và bit 1 nghĩa là input; vì vậy mask 0x00 đặt toàn bộ bốn bit LED thành output.

Mục đích của bước: Liên kết code C với đúng AXI GPIO của thiết kế và cài đặt thuật toán dịch LED 0001 → 0010 → 0100 → 1000 bằng driver chuẩn.

Thao tác thực hiện: Từ BSP sinh bởi platform, kiểm tra xparameters.h. Thiết kế hiện tại có AXI GPIO ở base address 0x41200000, high address 0x4120FFFF và GPIO width = 4. Tạo main.c, include xparameters.h, xgpio.h, xstatus.h và sleep.h; khởi tạo XGpio, đặt channel 1 là output và ghi lần lượt các mẫu LED.

Kiến thức rút ra: Qua bước này thấy rõ chuỗi software-to-hardware: macro địa chỉ sinh từ XSA → driver XGpio → ghi thanh ghi AXI GPIO → tín hiệu led_4bits trong PL. Đây là nội dung cốt lõi của hardware/software codesign trong Lab.

Tham số	Giá trị và ý nghĩa
XPAR_XGPIO_0_BASEADDR	0x41200000 - địa chỉ base của AXI GPIO
High address	0x4120FFFF - cuối vùng 64 KB
GPIO width	4 bit - tương ứng led_4bits[3:0]
LED_CHANNEL	1 - channel được dùng để xuất dữ liệu
Direction mask	0x00 - tất cả 4 bit là output

```

1 #include "xparameters.h"
2 #include "xgpio.h"
3 #include "xstatus.h"
4 #include "sleep.h"
5
6 #define GPIO_BASEADDR    XPAR_XGPIO_0_BASEADDR
7 #define LED_CHANNEL      1
8 #define LED_MASK         0x0F
9
10 int main(void)
11 {
12     XGpio Gpio;
13     int Status;
14     u32 led_value = 0x01;
15
16     Status = XGpio_Initialize(&Gpio, GPIO_BASEADDR);
17     if (Status != XST_SUCCESS)
18         return XST_FAILURE;
19
20     XGpio_SetDataDirection(&Gpio, LED_CHANNEL, 0x00);
21
22     while (1)
23     {
24         XGpio_DiscreteWrite(&Gpio, LED_CHANNEL, led_value);
25         sleep(1);
26         led_value <<= 1;
27         if (led_value > LED_MASK)
28             led_value = 0x01;
29     }
30
31     return XST_SUCCESS;
32 }

```

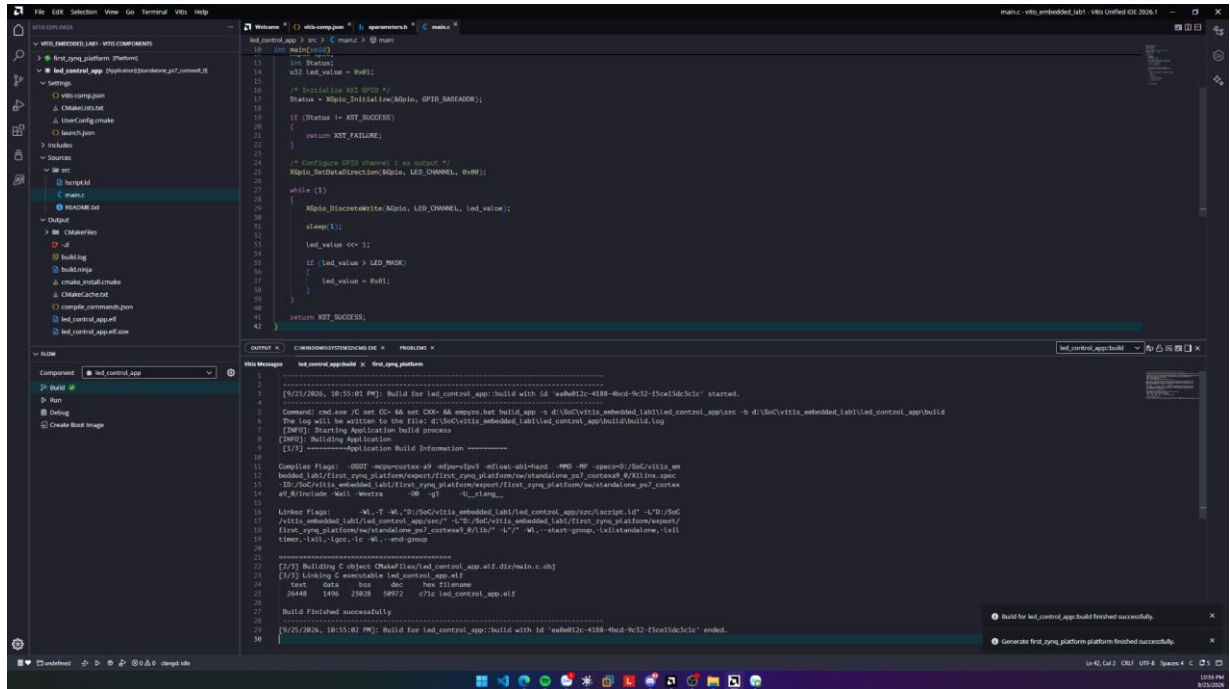
Giải thích chương trình: XGpio_Initialize tạo instance driver cho AXI GPIO dựa trên thông tin phần cứng sinh từ platform. XGpio_SetDataDirection với mask 0x00 đặt bốn chân của channel 1 ở chế độ output. Trong vòng lặp, XGpio_DiscreteWrite ghi led_value ra GPIO, sau đó dịch trái một bit. Khi giá trị vượt 0x0F, biến quay lại 0x01. Vì vậy chuỗi logic là 0001 → 0010 → 0100 → 1000 → lặp lại. sleep(1) tạo khoảng dừng một giây trong trường hợp chạy thực trên processor.

Bước 5 - Build application và kiểm tra file ELF

Cơ sở lý thuyết: Build application gồm compile source C thành object file và link với startup code, BSP, driver và linker script để tạo ELF. ELF chứa mã máy ARM, dữ liệu và thông tin cần thiết để debugger/loader nạp chương trình vào processor. Build thành công chứng minh source code, header, driver và linker configuration tương thích với platform.

Mục đích của bước: Kiểm tra chương trình có thể được dịch hoàn chỉnh cho Cortex-A9 và tạo sản phẩm thực thi led_control_app.elf.

Thao tác thực hiện: Chọn component led_control_app trong FLOW và chạy Build. Vitis compile main.c bằng toolchain Cortex-A9, sau đó link với BSP standalone và driver cần thiết. Kết quả cần quan sát là Build Finished successfully và file led_control_app.elf xuất hiện trong Output.



Hình 23. Build led_control_app thành công trong Vitis Unified IDE.

Kết quả và ý nghĩa: Ở Output có các dòng Building C object, Linking C executable led_control_app.elf và Build Finished successfully. Thanh trạng thái cũng không còn error/warning của build. File ELF là chương trình thực thi cho ARM Cortex-A9. Đây là kết quả tương đương Figure 1.28 “Build Finished Console Message” của tutorial.

Kiến thức rút ra: Bước này giúp phân biệt lỗi editor/indexer với lỗi compiler thật. Chỉ khi compile và link thành công mới có thể khẳng định application hợp lệ về mặt build; file ELF là đầu ra phần mềm tương đương vai trò bitstream ở phía hardware.

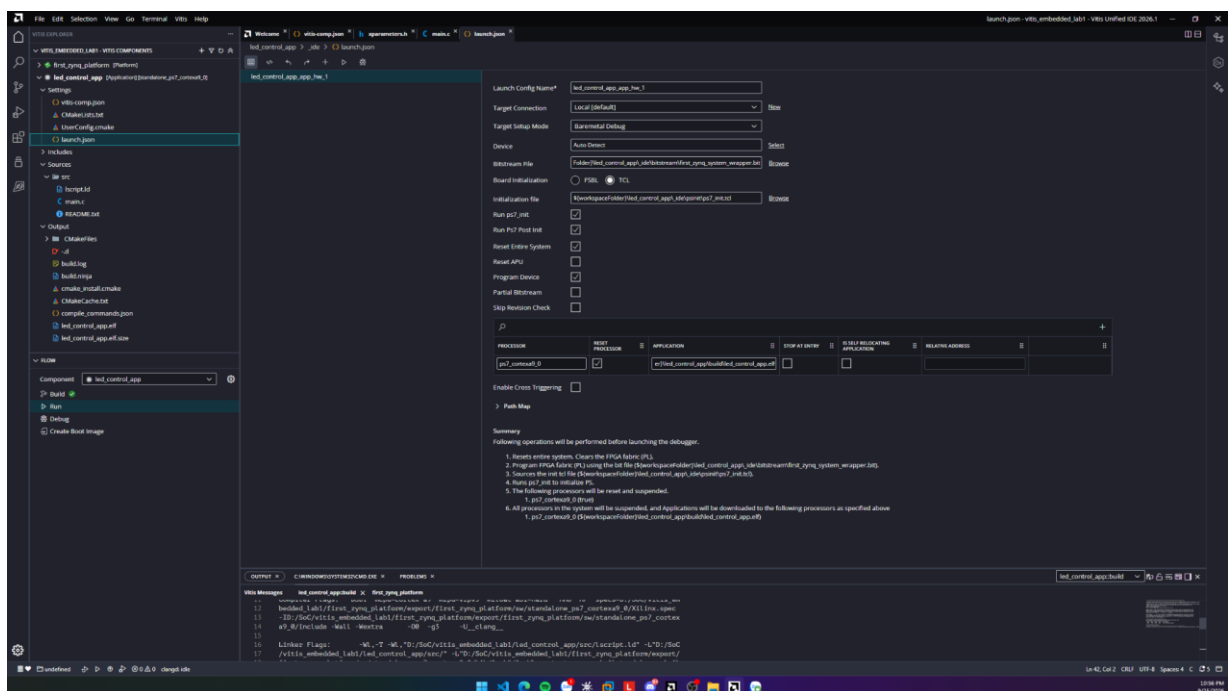
Bước 6 - Kiểm tra cấu hình Run và giới hạn do không có board

Cơ sở lý thuyết: Baremetal Debug trong Vitis thường sử dụng JTAG để reset PS, cấu hình PL bằng bitstream, chạy ps7_init và tải ELF vào Cortex-A9. Flow này cần một target vật lý được kết nối. QEMU chỉ mô phỏng được khi môi trường có machine/model phù

hợp; trong cài đặt hiện tại không có launcher Cortex-A9/Zynq-7000 hoàn chỉnh cho custom PL peripheral.

Mục đích của bước: Xác định chính xác lý do nút Run chưa thể hoàn thành và tránh diễn giải sai rằng lỗi nằm ở main.c hoặc AXI GPIO.

Thao tác thực hiện: Mở launch.json của application để kiểm tra cách Vitis định chạy chương trình. Cấu hình hiện tại có Target Setup Mode = Baremetal Debug, Target Connection = Local, bật Program Device, chạy ps7_init và download led_control_app.elf tới ps7_cortexa9_0. Đây là flow JTAG/hardware tương đương Run As → Launch on Hardware (GDB) của tutorial.



Hình 24. Launch configuration của led control app.

Kết quả và ý nghĩa: Hình cho thấy nút Run của Vitis hiện tại được cấu hình cho hardware target chứ không phải behavioral simulation. Khi không có ZC706 kết nối qua JTAG, Vitis không thể tìm ARM device để download ELF. Vì vậy báo cáo dừng ở build ELF đối với phần PS và không ghi nhận “LED vật lý đã chạy”. Hai bước Program FPGA và LED flashing của Figure 1.30-1.31 trong tutorial không được coi là kết quả thực nghiệm của bài này.

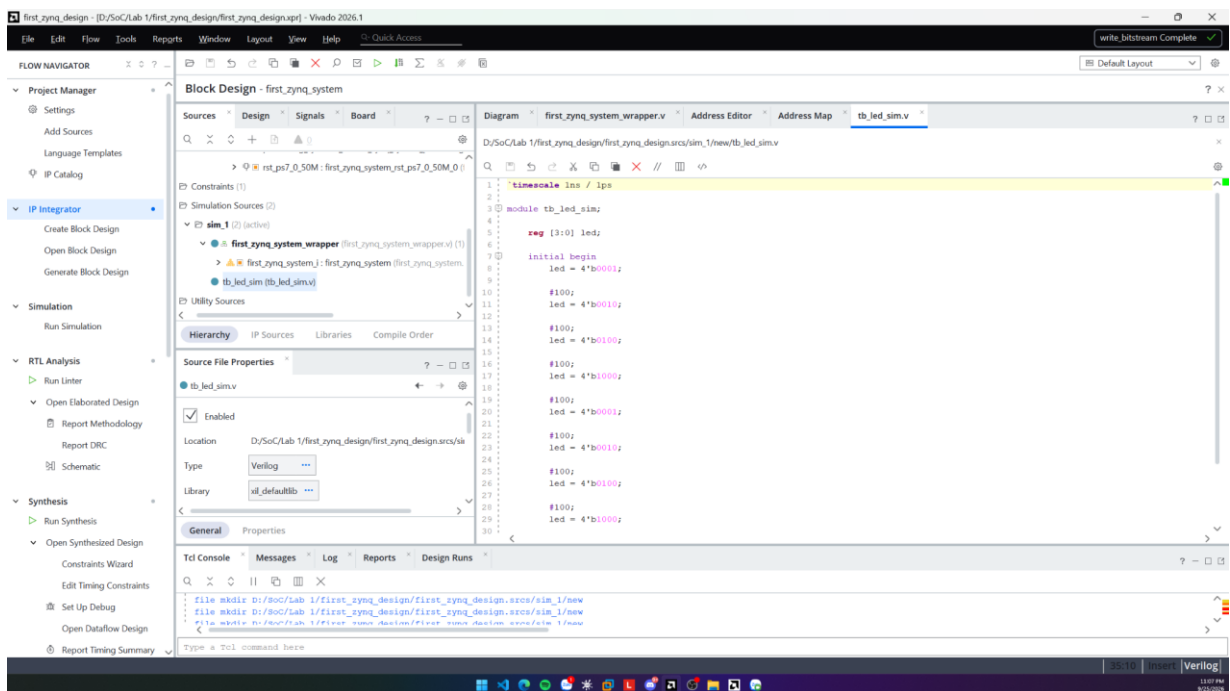
Kiến thức rút ra: Qua bước này học được rằng Build và Run là hai giai đoạn khác nhau. Build xác nhận chương trình được tạo thành ELF; Run còn phụ thuộc target execution environment, JTAG, board hoặc emulator hỗ trợ đúng phần cứng.

Bước 7 - Behavioral Simulation chuỗi LED 4-bit

Cơ sở lý thuyết: Behavioral Simulation kiểm tra hành vi tín hiệu theo mô hình HDL/testbench mà không cần board thật. Testbench chủ động tạo stimulus và theo dõi waveform. Trong bài này, testbench chỉ mô phỏng chuỗi trạng thái LED tương ứng thuật toán của main.c; nó không mô hình hóa Cortex-A9, driver XGpio hay transaction AXI đầy đủ.

Mục đích của bước: Bổ sung bằng chứng simulation-only cho chuỗi LED trong điều kiện không có ZC706 vật lý, đồng thời minh họa trực quan giá trị 4-bit thay đổi theo thời gian.

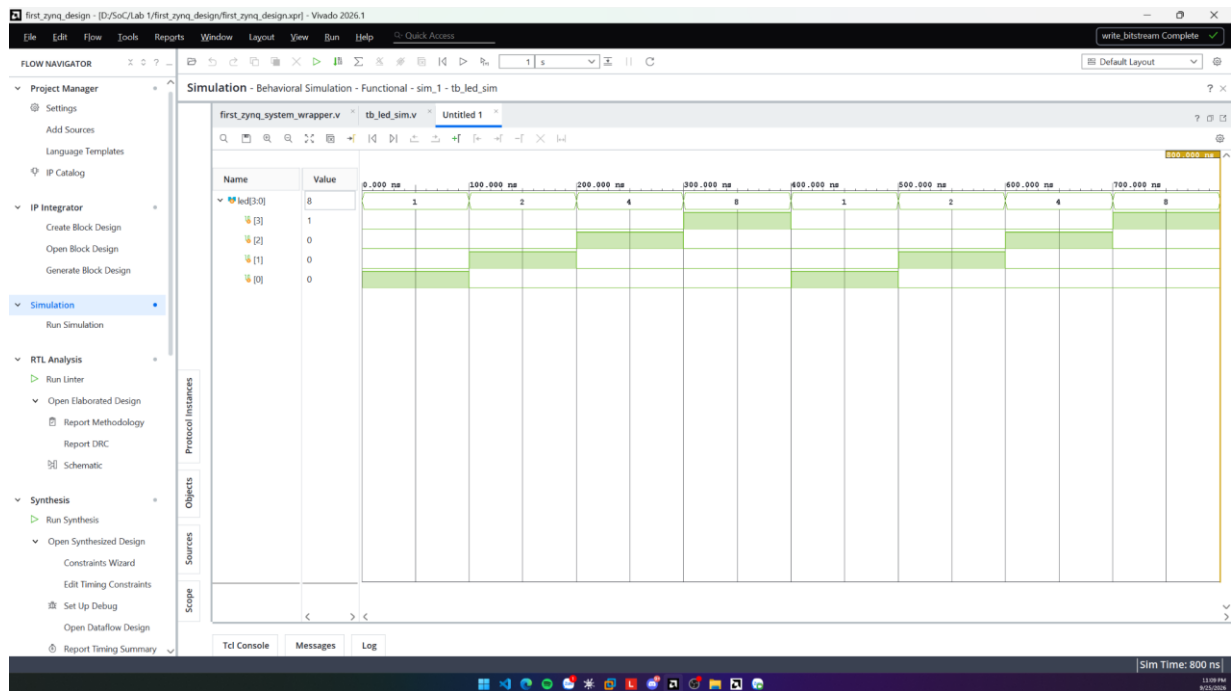
Thao tác thực hiện: Để có kết quả kiểm chứng trong điều kiện simulation-only, quay lại Vivado và tạo simulation source tb_led_sim.v. Testbench đặt led[3:0] lần lượt bằng 0001, 0010, 0100, 1000 với khoảng cách 100 ns, sau đó lặp lại. Đặt tb_led_sim làm simulation top và chọn Run Behavioral Simulation.



Hình 25. Testbench tb_led_sim.v dùng để tạo chuỗi trạng thái LED.

Kết quả và ý nghĩa: Testbench này chỉ mô phỏng hành vi của chuỗi LED tương ứng thuật toán trong main.c; nó không mô phỏng việc Cortex-A9 thực thi mã C hay giao dịch AXI ở mức processor. Các khoảng #100 ns được dùng để waveform dễ quan sát, không đại diện cho sleep(1) trong chương trình C.

Kiến thức rút ra: Bước này giúp phân biệt mô phỏng hành vi với mô phỏng toàn hệ thống. Waveform chứng minh pattern LED được mô tả đúng, nhưng không được dùng để tuyên bố rằng CPU đã thực thi ELF hoặc AXI GPIO đã được truy cập trong QEMU.



Hình 26. Kết quả Behavioral Simulation của `led[3:0]`.

Waveform hiển thị bus `led[3:0]` theo giá trị thập phân $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 8$, tương ứng nhị phân $0001 \rightarrow 0010 \rightarrow 0100 \rightarrow 1000$. Từng bit `led[0]`, `led[1]`, `led[2]`, `led[3]` lần lượt ở mức 1 trong từng khoảng thời gian, xác nhận đúng hành vi LED chạy 4 bit đã mô tả trong `main.c`.

Nhận xét và kết luận

Bài Lab đã hoàn thành chuỗi thiết kế hardware/software codesign từ kiến trúc Zynq PS-PL đến application bare-metal. Quan trọng hơn kết quả cuối cùng, từng bước cho thấy mối liên hệ xuyên suốt: Block Design xác lập kiến trúc; Address Editor tạo memory map; HDL Wrapper đưa kiến trúc vào flow RTL; Synthesis và Implementation biến mô tả logic thành phần cứng FPGA; Bitstream cấu hình PL; XSA chuyển giao thông tin phần cứng sang Vitis; BSP và `xparameters.h` phản ánh memory map cho phần mềm; cuối cùng driver `XGpio` cho phép code C tác động đến tín hiệu LED trong PL. Qua đó người thực hiện nắm được cách một lệnh phần mềm trên Cortex-A9 có thể điều khiển một ngoại vi tự thiết kế trong FPGA thông qua AXI.

Điểm khác so với Tutorial 1 là môi trường mới sử dụng Vivado/Vitis 2026.1, ZC706 4 LED và AXI SmartConnect thay cho ZedBoard 8 LED, SDK 2014.1 và AXI Interconnect. Do không có board thật, bước Program FPGA/Launch on Hardware của tutorial không được thực hiện; thay vào đó Behavioral Simulation được dùng để minh họa chuỗi LED 4-bit. Cách trình bày này giữ đúng phạm vi kết quả thực tế và tách rõ giữa “build phần mềm cho PS” với “mô phỏng hành vi LED”.

Hình nhóm:

